
RAG KMS

Полное руководство пользователя

Интеллектуальная система поиска по документам — версия 5.1

Версия 5.1 • 2026

Автор: Александр Князев,
начальник отдела технологий декарбонизации АО «НИИ НПО «ЛУЧ» — ПФ

Ubuntu · Python · Ollama · ChromaDB · Obsidian · Claude Code · MCP

PDF, DOCX, DOC, TXT, MD, XLSX, CSV, PPTX, ODT

Интеллектуальная система поиска по документам

Что уточнено в версии 5.1 (по опыту реальной установки и работы): требование Python 3.10–3.12 (на 3.13+ ChromaDB не собирается); реальная модель эмбедингов multilingual-e5 (по умолчанию e5-large ~2.2 ГБ; e5-small ~470 МБ — для скорости); фактические имена шаблонов и расположение заметок; корректные флаги (doc_to_obsidian.py --force вместо несуществующих); надёжная авто-обработка на USB/NTFS (опрос + догоняющее сканирование при переподключении, починка «грязного» NTFS через ntfsfix); автозапуск сервиса без входа в систему (linger); поведение «размышляющих» моделей (qwen3/deepseek-r1); опциональное GPU-ускорение индексации через NVIDIA CUDA (раздел 6.11), при этом чат использует CPU-встраивание, оставляя видеопамять под LLM. Скрипты системы обновлены соответственно.

Это руководство написано для людей, которые не называют себя «программистами» или «системными администраторами», но хотят иметь мощный инструмент для работы с большими архивами научных, технических и нормативных документов. Мы будем двигаться шаг за шагом, объясняя каждое действие и каждую команду простыми словами.

Оглавление

- Раздел 0. Что вы получите. Схема работы системы
- Раздел 1. Установка
 - о 1.1 Открываем терминал
 - о 1.2 Проверяем версию Ubuntu
 - о 1.3 Проверяем Python
 - о 1.4 Проверяем Ollama
 - о 1.5 Загружаем языковую модель
 - о 1.6 Настраиваем USB-диск (если архив на USB)
 - о 1.7 Если архив на системном диске (без USB)
 - о 1.8 Распаковываем архив системы
 - о 1.9 Создаём виртуальное окружение
 - о 1.10 Устанавливаем зависимости
- Раздел 2. Настройка
 - о 2.1 Настраиваем пути в config.py
 - о 2.2 Устанавливаем Obsidian
 - о 2.3 Настраиваем Obsidian Vault
 - о 2.4 Копируем шаблоны заметок
- Раздел 3. Первый запуск
 - о 3.1 Кладем первые документы

- о 3.2 Запускаем индексацию RAG
- о 3.3 Создаём заметки Obsidian
- о 3.4 Открываем граф в Obsidian
- о 3.5 Запускаем чат-интерфейс
- Раздел 4. Автозапуск наблюдателя
 - о 4.1 Что делает наблюдатель
 - о 4.2 Установка как сервис
 - о 4.3 Управление сервисом
 - о 4.4 Проверка работы наблюдателя
 - о 4.5 Работа с USB-диском
- Раздел 5. Ежедневная работа
 - о 5.1 Добавление новых документов
 - о 5.2 Работа с Obsidian
 - о 5.3 Вопросы через чат-интерфейс
 - о 5.4 Вопросы из командной строки
 - о 5.5 Обновление таксономии тегов
- Раздел 6. Устранение проблем
 - о 6.1 «command not found» при запуске python-скриптов
 - о 6.2 «Ollama недоступна» / «Connection refused»
 - о 6.3 «Модель не найдена» / «Model not found»
 - о 6.4 Ответ на русском плохого качества
 - о 6.5 Документ не индексируется
 - о 6.6 Наблюдатель не видит новые файлы
 - о 6.7 Нет места на диске
 - о 6.8 Забыл, как запустить чат
 - о 6.9 Ollama работает очень медленно
 - о 6.10 Ошибка при установке зависимостей
 - о 6.11 Ускорение индексации через видеокарту (GPU)
- Раздел 7. Управление LLM-провайдерами
 - о 7.1 Переключение провайдера
 - о 7.2 Рекомендуемые модели для нефтегазовой тематики
 - о 7.3 Переменные окружения
 - о 7.4 Устранение проблем с провайдерами
- Раздел 8. Обновление Ollama и LLM
- Раздел 9. Подключение Claude Code через MCP (опционально)
 - о 9.0 Что такое Claude Code и MCP
 - о 9.1 Установка Claude Code
 - о 9.2 Подготовка MCP-сервера
 - о 9.3 Подключение MCP к Claude Code
 - о 9.4 Первые запросы
 - о 9.5 Настройка CLAUDE.md
 - о 9.6 Использование инструментов напрямую
 - о 9.7 Устранение проблем

- о 9.8 Шпаргалка команд Claude Code
- Приложение А. Структура файлов системы
- Приложение Б. Глоссарий
- Приложение В. Проверочный список установки
- Приложение Г. Часто задаваемые вопросы (FAQ)
- Шпаргалка. Все ключевые команды

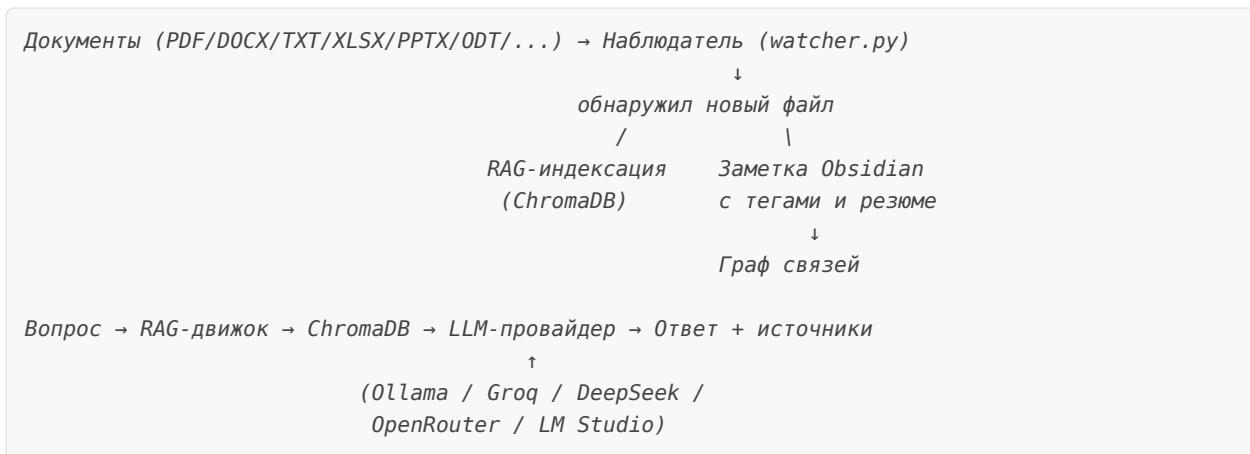
Раздел 0. Что вы получите. Схема работы системы

После того как вы пройдёте это руководство от начала до конца, у вас будет работающая система, которая умеет:

- **Автоматически обрабатывать новые документы** — достаточно скопировать файл в папку (или подключить USB-диск с архивом), и система сама всё сделает: извлечёт текст, проиндексирует его и создаст заметку. Поддерживаются форматы: PDF, DOCX, DOC, TXT, MD, XLSX, CSV, PPTX, ODT.
- **Отвечать на вопросы по содержанию ваших документов** — вы можете спросить что-то вроде «В каких регламентах описаны требования к опрессовке трубопровода?» и получить конкретный ответ со ссылками на источники.
- **Работать полностью офлайн** — вся обработка происходит на вашем компьютере, никакие данные не уходят в интернет, никакой подписки не требуется.
- **Организовывать знания в Obsidian** — каждый документ автоматически превращается в структурированную заметку с тегами, резюме и ссылками на связанные работы. Граф связей в Obsidian покажет, какие темы и авторы пересекаются в вашем архиве.
- **Задавать вопросы на русском и на английском** — языковая модель понимает оба языка и отвечает на том языке, на котором вы задаёте вопрос.
- **Работать с архивом на USB-диске** — если ваши документы хранятся на внешнем накопителе, система умеет работать с ним напрямую через символическую ссылку, не дублируя гигабайты данных на системный диск.
- **Использовать облачные и локальные LLM-провайдеры** — в версии 5.0 (v4.2) добавлена поддержка Groq, DeepSeek, OpenRouter и LM Studio в дополнение к локальной Ollama.

Схема работы системы

Прежде чем начать, полезно понять «большую картину» — как все части системы связаны между собой. Не переживайте, если что-то непонятно сейчас — к концу руководства всё станет на своё место.



Объяснение по частям:

- **Документы на USB** — ваши материалы в форматах PDF, DOCX, DOC, TXT, MD, XLSX, CSV, PPTX, ODT, хранящиеся на флешке или внешнем диске (или в обычной папке на компьютере).
- **Наблюдатель (watcher.py)** — небольшая программа, которая постоянно «смотрит» на вашу папку с архивом. Как только там появляется новый файл, она запускает обработку.
- **RAG-индексация (ChromaDB)** — RAG расшифровывается как *Retrieval-Augmented Generation* (поиск с дополненной генерацией). Грубо говоря, это технология, которая сначала ищет в вашей базе знаний самые релевантные фрагменты, а потом использует их для ответа. ChromaDB — это специальная база данных, которая умеет искать по смыслу, а не только по точным совпадениям слов.
- **Заметка Obsidian** — автоматически созданный структурированный документ в формате Markdown с метаданными, кратким резюме и тегами.
- **Граф связей** — визуальное отображение того, как заметки связаны друг с другом через общие темы, авторов, ключевые слова.
- **LLM-провайдер** — в версии 5.0 система поддерживает несколько провайдеров: локальную Ollama (без интернета), Groq (облако, бесплатно), DeepSeek, OpenRouter и LM Studio. Именно LLM формулирует ответы на человеческом языке.

Раздел 1. Установка

Этот раздел посвящён подготовке всего необходимого программного обеспечения. Мы пройдем через каждый шаг методично, и в конце у вас будет готовая к работе среда.

1.1 Открываем терминал

Что такое терминал?

Терминал (его ещё называют «консоль», «командная строка» или «shell») — это программа, которая позволяет управлять компьютером с помощью текстовых команд. Вместо того чтобы нажимать кнопки мышкой, вы вводите инструкции текстом. Звучит старомодно, но для задач

настройки и автоматизации это намного быстрее и точнее, чем графический интерфейс.

Как открыть терминал в Ubuntu:

Самый простой способ — нажать сочетание клавиш **Ctrl + Alt + T**. Должно появиться тёмное окно с мигающим курсором.

Если сочетание клавиш не работает, можно найти терминал вручную:

1. Нажмите клавишу **Super** (она же «Windows» на большинстве клавиатур) — откроется меню приложений.
2. Начните вводить слово «Терминал» или «Terminal».
3. Кликните на появившийся значок.

Что значит строка приглашения?

После открытия терминала вы увидите что-то похожее на:

```
username@computer:~$
```

Давайте разберём это по частям:

- username — имя вашей учётной записи в Ubuntu (то, что вы вводите при входе в систему).
- computer — имя вашего компьютера.
- ~ — это сокращение для вашей домашней директории (папки). Полный путь к ней обычно выглядит как /home/username. Символ ~ — это просто удобное сокращение.
- \$ — означает, что вы работаете как обычный пользователь (не администратор). Если бы вы работали как администратор (root), здесь был бы символ #.

[i] Совет

Не нужно вводить сам символ \$ при наборе команд — он уже стоит в строке приглашения. Вводите только то, что написано после него.

1.2 Проверяем версию Ubuntu

Для начала убедимся, что у вас подходящая версия операционной системы. Наша система работает на Ubuntu 20.04 и новее.

```
lsb_release -a
```

[>>] Что должно произойти

Что увидите:

```
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 22.04.3 LTS
Release:       22.04
Codename:      jammy
```

Нас интересует строка Release. Если там написано 20.04, 22.04 или 24.04 — всё отлично, продолжаем. Если у вас старее — рекомендуется обновить Ubuntu через официальные каналы, прежде чем продолжать.

[i] Совет

Буквы LTS в названии версии означают *Long-Term Support* — то есть эта версия получает обновления безопасности в течение 5 лет. Для рабочей системы это лучший выбор.

1.3 Проверяем Python

Что такое Python?

Python — это язык программирования, на котором написана большая часть нашей системы. Он очень популярен в научном сообществе и в мире искусственного интеллекта именно потому, что позволяет быстро писать работающий код. Для запуска Python-программ на вашем компьютере должен быть установлен интерпретатор Python — программа, которая «читает» код и выполняет его.

Проверим, что Python установлен и у него подходящая версия:

```
python3 --version
```

[>>] Что должно произойти

Что увидите:

```
Python 3.10.12
```

или похожую строку с версией 3.10, 3.11 или 3.12 — всё это подходит. Нужен Python **3.10-3.12**.

[!] Важно: на **Python 3.13 и новее** установка зависимостей пока падает — для свежего Python ещё нет готовых пакетов ChromaDB, и сборка chroma-hnswlib завершается ошибкой Python.h: No such file or directory. Если python3 --version показывает 3.13+, поставьте 3.12 рядом (sudo apt install python3.12 python3.12-venv) и создавайте окружение командой python3.12 -m venv .venv.

Если версия ниже 3.10 (например, 3.8 или 3.9):

Нужно установить более новую версию. В Ubuntu это делается так:

```
sudo apt update && sudo apt install python3.11 -y
```

Что такое sudo?

sudo — от английского *SuperUser DO* («сделай как суперпользователь»). Некоторые команды требуют прав администратора — например, установка новых программ. sudo временно даёт вашей команде эти права. При первом использовании система попросит ввести ваш пароль. Обратите внимание: когда вы вводите пароль в терминале, символы на экране не появляются — это нормально, так задумано для безопасности. Просто введите пароль и нажмите Enter.

[>>] Что должно произойти

Что увидите после `sudo apt update`: Длинный список строк вида `Get:1 http://... — система скачивает информацию о доступных пакетах. В конце — строка Reading package lists... Done.`

[>>] Что должно произойти

Что увидите после `sudo apt install python3.11 -y`: Процесс установки с прогресс-барами. Флаг `-y` означает «автоматически отвечать «да» на все вопросы установщика».

После установки убедитесь, что всё прошло успешно:

```
python3.11 --version
```

[>>] Что должно произойти

Что увидите:

```
Python 3.11.x
```

1.4 Проверяем Ollama

Что такое Ollama?

Ollama — это программа, которая позволяет запускать большие языковые модели (LLM) прямо на вашем компьютере, без интернета и без платных API. Она работает как «сервер» — запускается в фоне и отвечает на запросы от нашей системы.

Проверим, установлена ли Ollama:

```
ollama list
```

[>>] Что должно произойти

Что увидите, если Ollama установлена:

NAME	ID	SIZE	MODIFIED
mistral:latest	f974a74358d6	4.1 GB	2 weeks ago

или пустую таблицу с заголовками — это значит, что Ollama установлена, но модели ещё не скачаны.

[>>] Что должно произойти

Что увидите, если Ollama НЕ установлена:


```
bash: ollama: command not found
```

В этом случае переходите к установке ниже.

Если Ollama не установлена:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Разберём эту команду:

- curl — программа для скачивания файлов из интернета.
- -fsSL — набор флагов: -f (не показывать страницы ошибок), -s (тихий режим), -S (всё же показывать ошибки), -L (следовать за перенаправлениями).
- https://ollama.com/install.sh — адрес скрипта установки.
- | sh — символ | называется «pipe» (труба) и передаёт скачанный скрипт прямо на выполнение команде sh (shell-интерпретатор).

[>>] Что должно произойти

Что увидите:

```
>>> Installing ollama to /usr/local/bin...
>>> Creating ollama user...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
>>> Enabling and starting ollama service...
```

Установка займёт 1-2 минуты.

После установки проверьте ещё раз:

```
ollama list
```

[>>] Что должно произойти

Что увидите: Пустую таблицу с заголовками NAME ID SIZE MODIFIED — Ollama установлена и работает.

[!] Важно

После установки Ollama вам может потребоваться перезапустить терминал или выполнить команду `source ~/.bashrc`, чтобы команда `ollama` стала доступной в текущей сессии.

1.5 Загружаем языковую модель

Что такое LLM (большая языковая модель)?

LLM (*Large Language Model*) — это программа, которая обучена на огромных массивах текста и умеет понимать и генерировать человеческий язык. Именно LLM будет «думать» и формулировать ответы на ваши вопросы. Для начальной настройки мы будем использовать модель **Mistral** — это открытая модель, которая хорошо работает на обычных компьютерах и понимает русский язык. В дальнейшем вы сможете сменить модель (смотрите Раздел 7).

```
ollama pull mistral
```

Команда `pull` (от англ. «тянуть, скачивать») загружает модель из репозитория Ollama на ваш компьютер.

[i] Совет

Это нормально: модель весит около 4 ГБ, и скачивание займёт от 10 до 20 минут в зависимости от скорости вашего интернета. Вы увидите прогресс-бар вида:

```
pulling manifest
pulling ff82381e2bea... 100% ██████████ 3.8 GB
pulling 43070e2d4e53... 100% ██████████ 11 KB
verifying sha256 digest
writing manifest
removing any unused layers
success
```

Не закрывайте терминал и дождитесь слова `success`.

После скачивания проверим:

```
ollama list
```

[>>] Что должно произойти

Что увидите:

NAME	ID	SIZE	MODIFIED
mistral:latest	f974a74358d6	4.1 GB	Just now

Отлично! Модель готова к работе.

[i] Совет

Если у вас компьютер с ОЗУ 16 ГБ и больше, рекомендуется установить модель `qwen2.5:7b` — она лучше работает с русским языком и технической тематикой. Подробнее — в Разделе 7.

1.6 Настраиваем USB-диск (если архив на USB)

Если ваши файлы хранятся не на USB, а прямо на компьютере — пропустите этот пункт и перейдите к разделу 1.7.

Зачем нужна символическая ссылка?

Представьте, что ваш USB-диск — это книжная полка в другой комнате. Вместо того чтобы каждый раз ходить в ту комнату, вы можете повесить на стену рядом с собой табличку с указателем: «Книжная полка — там». Символическая ссылка (symlink) — это такой «указатель» в файловой системе. Папка `~/KMS/archive` будет выглядеть как обычная папка на вашем компьютере, но на самом деле это будет «указатель» на папку на вашем USB-диске.

Это удобно по двум причинам:

1. Программы могут работать с единым путём `~/KMS/archive`, не зная, где физически находятся файлы.
2. Вы не тратите место на системном диске — данные остаются на USB.

Шаг 1: Подключите USB-диск и узнайте его имя в системе.

```
lsblk
```

Команда `lsblk` (*list block devices*) показывает все подключённые блочные устройства (диски и разделы).

[>>] Что должно произойти

Что увидите:

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
sda	8:0	0	238.5G	0	disk	
└─sda1	8:1	0	512M	0	part	/boot/efi
└─sda2	8:2	0	238G	0	part	/
sdb	8:16	1	14.9G	0	disk	
└─sdb1	8:17	1	14.9G	0	part	/media/username/MY_USB

Ваш USB-диск обычно обозначается `sdb` или `sdc`. Обратите внимание на столбец `MOUNTPOINT` — там будет путь вроде `/media/username/MY_USB`. Это и есть то, что нам нужно.

Теперь посмотрим на содержимое папки с подключёнными дисками:

```
ls /media/$USER/
```

Здесь `$USER` — специальная переменная, которая автоматически подставляет имя вашего пользователя.

[>>] Что должно произойти

Что увидите:

```
MY_USB
```

или другое имя вашего диска. Запомните это имя — оно нам понадобится.

Шаг 2: Создайте структуру папок и символическую ссылку.

Сначала создадим папку KMS в домашней директории (если её ещё нет):

```
mkdir -p ~/KMS
```

Флаг -p означает «создать все промежуточные папки, если их нет, и не выдавать ошибку, если папка уже существует».

Теперь создадим на USB-диске папку для архива (если её ещё нет) и символическую ссылку:

```
mkdir -p /media/$USER/ИМЯ_ДИСКА/petroleum_papers  
ln -s /media/$USER/ИМЯ_ДИСКА/petroleum_papers ~/KMS/archive
```

[!] Важно

Замените ИМЯ_ДИСКА на реальное имя вашего USB-диска (то, что вы увидели командой `ls /media/$USER/`). И замените `petroleum_papers` на имя папки с вашими документами, если она уже существует на диске. Например, если ваши PDF лежат в корне диска в папке `papers`, команда будет:

```
ln -s /media/$USER/MY_USB/papers ~/KMS/archive
```

Проверим, что ссылка создана корректно:

```
ls -la ~/KMS/archive
```

[>>] Что должно произойти

Что увидите:

```
lrwxrwxrwx 1 username username 35 Jan 15 10:23 /home/username/KMS/archive ->  
/media/username/MY_USB/petroleum_papers
```

Стрелочка -> показывает, куда ведёт символическая ссылка. Если вы видите эту стрелочку — всё правильно.

[!] Важно

Символическая ссылка будет «сломана», если USB-диск не подключён. Команда `ls ~/KMS/archive` выдаст ошибку. Это нормальное поведение — просто подключите диск.

1.7 Если архив на системном диске (без USB)

Если ваши документы хранятся прямо на компьютере (или вы хотите сначала попробовать систему без USB), просто создайте папку для архива:

```
mkdir -p ~/KMS/archive
```

Готово! Никаких символических ссылок не нужно — просто будете класть документы прямо в эту папку.

1.8 Распаковываем архив системы

Что такое tar.gz?

Когда вы скачиваете программу в Linux, она часто приходит в виде файла с расширением `.tar.gz`. Это сжатый архив — аналог `.zip` в Windows, только используются другие алгоритмы. `tar` — утилита для упаковки множества файлов в один, `.gz` — дополнительное сжатие алгоритмом `gzip`.

Предполагается, что вы скачали файл `rag_system_final.tar.gz` в папку Загрузки (`~/Загрузки` на русскоязычном Ubuntu или `~/Downloads` на англоязычном).

Перейдём в домашнюю директорию и распакуем только папку системы в `~/KMS/`:

```
cd ~  
tar -xzf ~/Загрузки/rag_system_final.tar.gz -C ~/KMS/ rag_system
```

Разберём флаги команды `tar`:

- `-x` — *extract* (извлечь файлы из архива)
- `-z` — использовать `gzip` для распаковки (нужен для `.gz`-файлов)
- `-f` — следующий аргумент — это имя файла архива
- `-C ~/KMS/` — *change directory* (распаковать в эту папку, а не в текущую)

[!] Важно

Если папка `~/Загрузки` не находит файл, попробуйте `~/Downloads/rag_system_final.tar.gz` — имя папки зависит от языка вашей системы.

Проверим, что распаковка прошла успешно:

```
ls ~/KMS/rag_system/
```

[>>] Что должно произойти

Что увидите:

```
chat_ui.py      config.py      ingest.py  
doc_to_obsidian.py pdf_to_obsidian.py rag_engine.py  
llm_provider.py requirements.txt install_service.sh  
watcher.py     obsidian_templates/ chroma_db/  
logs/          mcp_rag_server.py
```

Если вы видите эти файлы — архив распакован правильно. Каждый из этих файлов — это часть системы, с которой мы будем работать в этом руководстве. Обратите внимание на `llm_provider.py` — это новый модуль версии 5.0, который управляет несколькими

провайдерами LLM.

[i] Совет

Все примеры в руководстве предполагают, что система установлена в `~/KMS/rag_system`. Команда выше распаковывает её именно туда. Если вы распаковали архив в другое место — пути к архиву документов и заметкам всё равно вычисляются от вашей домашней директории (`~/KMS/archive` и `~/KMS/notes`), так что система продолжит работать (см. раздел 2.1).

1.9 Создаём виртуальное окружение

Что такое виртуальное окружение и зачем оно нужно?

Представьте, что Python — это мастерская, а пакеты (библиотеки) — это инструменты. Если разные проекты нуждаются в разных версиях одних и тех же инструментов, они могут мешать друг другу. Виртуальное окружение — это как отдельный «чемоданчик с инструментами» для каждого проекта. В нём хранятся именно те версии пакетов, которые нужны этому конкретному проекту, и они не конфликтуют с другими проектами или системными пакетами Python.

Перейдём в папку с системой и создадим виртуальное окружение:

```
cd ~/KMS/rag_system
python3 -m venv .venv
```

Разберём команду:

- `python3 -m venv` — запустить модуль `venv` (встроенный в Python инструмент для создания виртуальных окружений).
- `.venv` — имя папки, в которую будут установлены все файлы окружения. Точка в начале означает, что это «скрытая» папка (она не будет показана обычным `ls`, но `ls -a` её покажет).

Теперь активируем (включим) виртуальное окружение:

```
source .venv/bin/activate
```

[>>] Что должно произойти

Что увидите: В начале строки приглашения появится `(.venv)`:

```
(.venv) username@computer:~/KMS/rag_system$
```

Это значит, что виртуальное окружение активно и все последующие команды Python и `pip` будут работать внутри него.

[!] Важно

Виртуальное окружение нужно **активировать при каждом новом открытии терминала**, прежде чем запускать программы системы. Команда активации:

```
source ~/KMS/rag_system/.venv/bin/activate
```

Если вы забудете это сделать, увидите ошибки вида `ModuleNotFoundError: No module named 'chromadb'`. Это не поломка системы — просто нужно активировать окружение.

[i] Совет

Чтобы выйти из виртуального окружения, введите `deactivate`. Но во время работы с системой выходить из окружения не нужно.

1.10 Устанавливаем зависимости

Что такое зависимости и `pip`?

Наша система написана на Python и использует множество готовых библиотек (пакетов) — например, ChromaDB для работы с базой данных, Gradio для веб-интерфейса чата, python-docx для работы с Word-документами. Список всех нужных библиотек записан в файле `requirements.txt`.

Сначала установим системные пакеты — они нужны для обработки форматов DOC (старые Word-файлы):

```
sudo apt update
sudo apt install libreoffice antiword -y
```

Что делают эти пакеты:

- `libreoffice` — офисный пакет. Система использует его в фоновом режиме для конвертации старых `.doc`-файлов в текст. Не нужно запускать LibreOffice вручную.
- `antiword` — запасной конвертер `.doc`, используется если LibreOffice недоступен.

Установка займёт 3-5 минут — LibreOffice довольно большой пакет (~300 МБ).

`pip` — это менеджер пакетов Python (от *Pip Installs Packages*). Он умеет скачивать библиотеки из интернета и устанавливать их в нужное место.

Сначала обновим сам `pip` до последней версии:

```
pip install --upgrade pip
```

[>>] Что должно произойти

Что увидите:

```
Requirement already satisfied: pip in ./venv/lib/python3.10/site-packages (23.0.1)
Collecting pip
  Downloading pip-24.0-py3-none-any.whl (2.1 MB)
...
Successfully installed pip-24.0
```

Теперь установим все зависимости из файла требований:

```
pip install -r requirements.txt
```

[i] Совет

Это нормально: Вы увидите длинный список пакетов, которые скачиваются и устанавливаются. Процесс займёт от 5 до 15 минут в зависимости от скорости интернета. Не пугайтесь, если список кажется огромным — это стандартное поведение. Строки будут выглядеть примерно так:

```
Collecting chromadb==0.4.22
  Downloading chromadb-0.4.22-py3-none-any.whl (512 kB)
----- 512.3/512.3 kB 3.2 MB/s eta 0:00:00
Collecting langchain==0.1.0
  Downloading langchain-0.1.0-py3-none-any.whl (800 kB)
...
Successfully installed chromadb-0.4.22 langchain-0.1.0 ...
```

Когда всё установится, последней строкой будет `Successfully installed ...` с перечислением всех установленных пакетов.

[!] Важно

Если в процессе установки появляется строка с `ERROR` или `error`, прочитайте её внимательно. Часто это означает, что не хватает какой-то системной библиотеки. Наиболее частая ошибка решается командой:

```
sudo apt install python3-dev build-essential libssl-dev libffi-dev -y
```

После чего повторите `pip install -r requirements.txt`.

Раздел 2. Настройка

После установки всех компонентов нам нужно настроить систему под ваше конкретное окружение: указать, где лежат файлы, установить Obsidian и подготовить хранилище заметок.

2.1 Настраиваем пути в config.py

Что такое `nanop`?

nano — это простой текстовый редактор, который работает прямо в терминале. В отличие от vim или emacs, с ним легко разобраться без опыта. Управление осуществляется с помощью сочетаний клавиш, которые всегда показаны внизу экрана (символ ^ означает клавишу Ctrl).

Откроем файл конфигурации:

```
nano ~/KMS/rag_system/config.py
```

[>>] Что должно произойти

Что увидите: Текстовый редактор откроется прямо в терминале. Вверху — название файла, в середине — его содержимое, внизу — подсказки по управлению. Стрелками на клавиатуре можно перемещаться по файлу.

Чтобы быстро найти нужное место, используйте поиск: нажмите **Ctrl+W** (W — от *Where*), введите LLM_PROVIDER и нажмите Enter. Курсор переместится к первому вхождению этого слова.

Вручную нужно отредактировать только **три строки** — выбор провайдера и модели LLM:

```
LLM_PROVIDER: str = "ollama"  
LLM_MODEL:    str = "qwen3.5:latest" # уже установленная модель (или "mistral", если  
    скачали её)  
LLM_API_KEY:  str = ""
```

Что означает каждая строка:

- LLM_PROVIDER — активный провайдер LLM (подробнее — в Разделе 7).
- LLM_MODEL — имя модели для выбранного провайдера. По умолчанию в config.py указана qwen3.5:latest — модель, уже установленная в этой системе. Если в разделе 1.5 вы скачали mistral, укажите "mistral".
- LLM_API_KEY — API-ключ для облачных провайдеров (оставьте пустым при работе с Ollama).

[!] Важно

(о путях): Папки с документами и заметками **не нужно** прописывать вручную — система вычисляет их автоматически от вашей домашней директории:

- архив документов ~/KMS/archive
- заметки Obsidian ~/KMS/notes

В коде за это отвечают KMS_ARCHIVE_DIR и KMS_VAULT_DIR. Не редактируйте их и не превращайте в относительные пути вроде "KMS/archive" — иначе наблюдатель будет следить не за той папкой.

Если ваш архив лежит на USB-диске по нестандартному пути (и вы не делали символическую ссылку из раздела 1.6), можно явно указать корневую папку KMS через переменную окружения. Добавьте в конец ~/.bashrc:

```
export KMS_HOME="/media/$USER/ИМЯ_ДИСКА/KMS"
```

и выполните `source ~/.bashrc`.

Для редактирования просто переместите курсор к нужной строке стрелками и внесите изменения.

Сохранение и выход из nano:

1. Нажмите **Ctrl+X** (выход)
2. Появится вопрос `Save modified buffer?` — нажмите **Y** (да)
3. Появится строка с именем файла — нажмите **Enter** для подтверждения

[>>] Что должно произойти

Что увидите после сохранения: Nano закроется, и вы вернётесь в обычную командную строку терминала.

[i] Совет

Если вы случайно что-то испортили и хотите выйти БЕЗ сохранения изменений, нажмите **Ctrl+X**, затем **N** (нет). Файл останется нетронутым.

Проверка путей. Убедитесь, что система видит правильные папки. Выполните:

```
cd ~/KMS/rag_system && source .venv/bin/activate
python -c "import config; print('архив :', config.KMS_ARCHIVE_DIR); print('заметки:', config.KMS_VAULT_DIR)"
```

[>>] Что должно произойти

Что увидите:

```
архив : /home/ВАШЕ_ИМЯ/KMS/archive
заметки: /home/ВАШЕ_ИМЯ/KMS/notes
```

Если вместо этого вы видите путь с двойным KMS (`.../KMS/KMS/archive`) или относительный путь — у вас старая версия `config.py` с ошибкой вычисления путей.

[!] Важно

Если пути неверные — обновите `config.py`: в исправленной версии каталог KMS привязан к домашней директории (`$HOME/KMS`), а не к месту установки `rag_system`. Замените блок «Интеграция с Obsidian / KMS» в `config.py` на:

```
KMS_DIR = Path(os.environ.get("KMS_HOME", str(Path.home() / "KMS")))
KMS_ARCHIVE_DIR = KMS_DIR / "archive"
KMS_VAULT_DIR = KMS_DIR / "notes"
ARCHIVE_DIR: str = str(KMS_ARCHIVE_DIR)
OBSIDIAN_VAULT_DIR: str = str(KMS_VAULT_DIR)
```

(раньше там было `KMS_DIR = BASE_DIR.parent / "KMS"` и относительные строки `"KMS/archive"/"KMS/notes"`). После правки повторите проверку выше.

2.2 Устанавливаем Obsidian

Что такое Obsidian?

Obsidian — это приложение для создания и организации заметок в формате Markdown (простой текст с разметкой). Его главная особенность — умение строить «граф знаний»: визуальную карту связей между вашими заметками. В нашей системе каждый документ автоматически превращается в заметку Obsidian, и вы сможете видеть, как документы связаны между собой по темам и авторам.

Скачайте Obsidian с официального сайта: <https://obsidian.md>

На странице загрузки выберите вариант для Linux — файл будет называться примерно `obsidian-1.5.3.deb`. Сохраните его в папку Загрузки.

Установим скачанный `.deb`-файл (`.deb` — это формат пакетов для Ubuntu/Debian):

```
cd ~/Загрузки
sudo dpkg -i obsidian-*.deb
```

Команда `dpkg -i` (*install*) устанавливает пакет. Символ `*` означает «любые символы» — так мы не зависим от точного номера версии в имени файла.

[>>] Что должно произойти

Что увидите:

```
Selecting previously unselected package obsidian.
(Reading database ... 312840 files and directories currently installed.)
Preparing to unpack obsidian-1.5.3.deb ...
Unpacking obsidian (1.5.3) ...
Setting up obsidian (1.5.3) ...
```

Если появляется ошибка:

Иногда при установке `.deb`-пакетов Ubuntu сообщает об отсутствующих зависимостях. Это выглядит как строки с `Depends:`. В этом случае выполните:

```
sudo apt install -f
```

Флаг `-f` означает *fix broken* (исправить сломанное) — `apt` автоматически скачает и установит все недостающие зависимости и завершит установку Obsidian.

После установки Obsidian должен появиться в меню приложений Ubuntu. Нажмите Super (Win) и начните вводить «Obsidian» — вы должны увидеть его значок.

2.3 Настраиваем Obsidian Vault

Что такое Vault в Obsidian?

Vault (хранилище) — это просто папка на вашем компьютере, которую Obsidian отслеживает и в которой хранит все заметки. Obsidian может работать с несколькими хранилищами, но нам нужно создать одно — ~/KMS/notes.

Сначала создадим папку для хранилища:

```
mkdir -p ~/KMS/notes
```

Теперь откройте Obsidian. При первом запуске появится экран приветствия:

1. Нажмите **«Open folder as vault»** (Открыть папку как хранилище).
2. В диалоге выбора папки перейдите в домашнюю директорию, затем в KMS, затем выберите папку notes.
3. Нажмите **«Open»** (Открыть).

Obsidian откроется с пустым хранилищем. Слева вы увидите панель с файлами (пока пустую), справа — область для написания заметок.

[i] Совет

Если система не находит папку ~/KMS/notes в диалоге, попробуйте ввести путь вручную в строке адреса файлового диалога: нажмите Ctrl+L (или Ctrl+Shift+G на некоторых системах) и введите /home/ВАШЕ_ИМЯ/KMS/notes.

2.4 Копируем шаблоны заметок

Наша система использует шаблоны Obsidian — подготовленные структуры для заметок, которые автоматически заполняются данными из документов. Скопируем их в правильное место:

```
mkdir -p ~/KMS/notes/Templates
cp ~/KMS/rag_system/obsidian_templates/* ~/KMS/notes/Templates/
```

Проверим, что шаблоны скопировались:

```
ls ~/KMS/notes/Templates/
```

[>>] Что должно произойти

Что увидите:

```
'Literature Note.md'  'RAG Query.md'  'Research Session.md'  'Shell Commands.md'
```

Теперь нужно сказать Obsidian, где искать шаблоны:

1. В Obsidian откройте **Настройки** (Settings) — нажмите иконку шестерёнки в левом нижнем углу или Ctrl+,,.

2. В левом меню настроек перейдите в раздел **Core plugins** (Основные плагины).
3. Найдите плагин **Templates** и убедитесь, что он включён (переключатель должен быть синим).
4. Нажмите на иконку настроек рядом с Templates.
5. В поле **Template folder location** введите: Templates
6. Закройте настройки.

[i] Совет

Это нормально: Если вы видите в левой панели Obsidian папку Templates — всё настроено правильно.

Раздел 3. Первый запуск

Все компоненты установлены и настроены. Пришло время запустить систему впервые! Давайте добавим несколько документов, создадим индекс и проверим, что всё работает.

3.1 Кладём первые документы

Для первого запуска добавьте 2-3 файла в папку архива. Система поддерживает форматы: PDF, DOCX, DOC, TXT, MD, XLSX, CSV, PPTX, ODT. Это можно сделать двумя способами.

Способ 1: Через файловый менеджер (Nautilus)

Откройте файловый менеджер Ubuntu (значок папки на панели задач или Ctrl+L введите путь). Найдите ваши файлы, выделите их и перетащите в папку ~/KMS/archive. Всё!

Способ 2: Через терминал

```
cp /путь/к/файлу.docx ~/KMS/archive/
```

Например, если файл лежит в папке Загрузки:

```
cp ~/Загрузки/my_report.pdf ~/KMS/archive/  
cp ~/Загрузки/notes.docx ~/KMS/archive/  
cp ~/Загрузки/data.xlsx ~/KMS/archive/
```

Или скопировать все файлы одного типа сразу:

```
cp ~/Загрузки/*.pdf ~/KMS/archive/  
cp ~/Загрузки/*.docx ~/KMS/archive/
```

[i] Совет

Файлы разных форматов можно смешивать в одной папке и в подпапках — система обрабатывает каждый файл нужным способом автоматически.

Проверим, что файлы попали куда надо:

```
ls ~/KMS/archive/
```

[>>] Что должно произойти

Что увидите:

```
article_1.pdf  report.docx  data.xlsx
```

3.2 Запускаем индексацию RAG

Теперь попросим систему обработать документы: извлечь текст, разбить на фрагменты и создать «векторные представления» (embeddings — числовые коды, которые передают смысл текста) для последующего поиска.

Убедимся, что мы в правильной папке и виртуальное окружение активно:

```
cd ~/KMS/rag_system
source .venv/bin/activate
```

Запускаем индексацию:

```
python ingest.py
```

[i] Совет

Это нормально: При **первом** запуске система скачает многоязычную модель эмбедингов `intfloat/multilingual-e5-large` (~2.2 ГБ, хорошо понимает русский, высокое качество поиска). Скачивается один раз, далее берётся из кеша; на медленном интернете загрузка займёт 15-30 минут. Вы увидите:

```
Downloading intfloat/multilingual-e5-large...
Downloading: 100%|████████████████████| 2.24G/2.24G [18:40<00:00, 2.0MB/s]
```

[i] Совет: если важна скорость или экономия места, можно указать меньшую модель — `EMBEDDING_MODEL = "intfloat/multilingual-e5-small"` (~470 МБ): чуть ниже качество поиска, но заметно быстрее. После смены модели обязательно переиндексируйте: `python ingest.py --reset`.

[i] Устройство для эмбедингов. По умолчанию модель работает на CPU (`EMBEDDING_DEVICE = "cpu"` в `config.py`). Это сделано намеренно: на GPU с 8 ГБ памяти одновременно не

помещаются локальная LLM (Ollama, ~6 ГБ) и модель эмбедингов (~2 ГБ) — индексация падала с CUDA out of memory. На CPU конфликта нет, а эмбединг одного запроса всё равно занимает доли секунды (медленнее только массовая индексация). Если GPU свободен или его памяти достаточно — поставьте `EMBEDDING_DEVICE = "cuda"` (или `None` для авто-выбора) для ускорения индексации.

После скачивания модели начнётся обработка файлов:

[>>] Что должно произойти

Что увидите:

```
Processing: article_1.pdf [PDF]
  Extracting text... done (47 pages)
  Splitting into chunks... done (312 chunks)
  Creating embeddings... 100%|#####| 312/312 [00:45<00:00, 6.9it/s]
  Stored in ChromaDB.
Processing: report.docx [DOCX]
  Extracting text... done (12 pages)
  Splitting into chunks... done (78 chunks)
  ...

Indexing complete. Total documents: 3, Total chunks: 468
```

Каждый «chunk» (фрагмент) — это небольшой кусочек текста из документа (обычно 300-500 слов), которому система присваивает числовое представление. При поиске система найдёт наиболее релевантные фрагменты и передаст их языковой модели.

[i] Совет

Если индексация занимает очень долго (более 30 минут на один документ), скорее всего, система работает без поддержки GPU. Это нормально — просто медленнее. Можно оставить процесс работать и заняться другими делами, он не требует вашего внимания.

3.3 Создаём заметки Obsidian

Теперь попросим систему создать структурированные заметки в Obsidian для каждого проиндексированного документа. В v5.0 за это отвечает скрипт `doc_to_obsidian.py` — он умеет работать со всеми поддерживаемыми форматами.

Сначала запустим в режиме «сухого прогона» — это покажет, что будет создано, без реального создания файлов:

```
python doc_to_obsidian.py --dry-run
```

[>>] Что должно произойти

Что увидите:

DRY RUN MODE - no files will be created

*Would create: ~/KMS/notes/Articles/article_1.md
Title: Enhanced Oil Recovery Using CO2 Injection...
Type: PDF
Tags: #auto/co2-injection #auto/eor #auto/reservoir-simulation*

*Would create: ~/KMS/notes/Articles/report.md
Title: Quarterly Report Q1 2024
Type: DOCX
Tags: #auto/report #auto/quarterly*

Total: 3 notes would be created.

Если всё выглядит правильно, запускаем реальное создание заметок:

```
python doc_to_obsidian.py
```

[>>] Что должно произойти

Что увидите:

*Processing article_1.pdf [PDF]...
Generating summary via LLM... done
Extracting metadata... done
Creating note: ~/KMS/notes/Articles/article_1.md
Created*

*Processing report.docx [DOCX]...
Generating summary via LLM... done
...*

All done! Created 3 notes in ~/KMS/notes/Articles/

Создание заметок занимает около 1-2 минут на документ — в это время языковая модель генерирует краткое резюме содержания на основе извлечённого текста.

[i] Совет

Флаг `--dry-run` (от англ. «сухой прогон») — это очень полезный инструмент. Многие программы поддерживают его. Он позволяет «проиграть» операцию и посмотреть, что произойдёт, прежде чем реально что-то изменить. Полезно использовать перед операциями, которые сложно отменить.

3.4 Открываем граф в Obsidian

Теперь откройте Obsidian. В левой панели вы должны увидеть новую папку Articles с заметками для каждого документа.

Чтобы открыть граф связей:

1. Нажмите на значок **графа** в левой панели (иконка с точками, соединёнными линиями) — или используйте горячую клавишу **Ctrl+G**.
2. Откроется интерактивный граф.

[>>] Что должно произойти

Что увидите: Граф с несколькими узлами (точками) — по одному на каждую заметку — и линиями, соединяющими их через общие теги и упоминания. Чем больше у вас документов, тем интереснее будет выглядеть граф: появятся «кластеры» по темам, центральные узлы (часто цитируемые авторы или ключевые темы) и периферийные документы.

Вы можете:

- **Кликнуть на узел** — откроется заметка.
- **Потянуть узел** — переместить его на графе.
- **Прокрутить колёсо мыши** — приблизить/отдалить.
- **Правой кнопкой по пустому месту** — открыть параметры отображения.

3.5 Запускаем чат-интерфейс

Финальный шаг — запуск веб-интерфейса для общения с вашим архивом:

```
python chat_ui.py
```

[>>] Что должно произойти

Что увидите:

Running on local URL: `http://127.0.0.1:7860`

To create a public link, set `'share=True'` in `'launch()'`.

Откройте браузер (Firefox, Chrome — любой) и перейдите по адресу:

```
http://127.0.0.1:7860
```

[>>] Что должно произойти

Что увидите в браузере: Простой чат-интерфейс. Вверху — поле для ввода вопроса, ниже — история диалога. Возможно, также есть ползунок top-k — он управляет тем, сколько фрагментов из базы знаний система использует для ответа (чем больше, тем полнее ответ, но медленнее).

Попробуйте задать первый вопрос! Например:

- «О каких темах говорится в моих документах?»
- «Перечисли ключевые выводы из отчётов»
- «What is the main conclusion of the articles about CO2 injection?»

[i] Совет

Это нормально: Первый ответ может появиться через 15-30 секунд — модель «разогревается». Последующие ответы будут быстрее (5-15 секунд).

[!] Важно

Чат-интерфейс работает, пока открыт терминал с командой `python chat_ui.py`. Не закрывайте этот терминал! Если нужно сделать что-то ещё в терминале — откройте **новое** окно терминала (`Ctrl+Alt+T`).

Раздел 4. Автозапуск наблюдателя

4.1 Что делает наблюдатель

Наблюдатель (`watcher.py`) — это программа, которая постоянно «смотрит» на вашу папку с архивом. Представьте охранника, который дежурит у входа в библиотеку. Как только приносят новую книгу, он немедленно её регистрирует: вносит в каталог (ChromaDB) и создаёт карточку (заметку в Obsidian). Вам не нужно запускать `ingest.py` и `doc_to_obsidian.py` вручную каждый раз — наблюдатель делает это автоматически.

Технически наблюдатель работает как **системный сервис** (*systemd service*). Systemd — это система управления сервисами в Linux: она умеет запускать программы при старте компьютера, перезапускать их при ошибках и вести логи их работы.

4.2 Установка как сервис

Для установки наблюдателя как системного сервиса используем готовый скрипт:

```
cd ~/KMS/rag_system
bash install_service.sh
```

Что происходит автоматически во время выполнения скрипта:

1. Создаётся файл конфигурации сервиса в `~/config/systemd/user/rag-watcher.service`
2. Systemd перечитывает список сервисов (`systemctl --user daemon-reload`)
3. Сервис включается (разрешается его автозапуск при входе в систему)
4. Сервис немедленно запускается

[>>] Что должно произойти

Что увидите:

```
Installing RAG Watcher service...
Created: ~/.config/systemd/user/rag-watcher.service
Reloading systemd daemon...
Enabling service...
Starting service...

❑ Service installed and running!
Check status with: systemctl --user status rag-watcher
```

4.3 Управление сервисом

Теперь у вас есть набор команд для управления наблюдателем:

Проверить, работает ли сервис:

```
systemctl --user status rag-watcher
```

[>>] Что должно произойти

Что увидите, если сервис работает:

```
● rag-watcher.service - RAG Watcher Service
   Loaded: loaded (/home/username/.config/systemd/user/rag-watcher.service; enabled)
   Active: active (running) since Mon 2024-01-15 10:23:45 MSK; 5min ago
   Main PID: 12345 (python)
   ...
```

Ключевое слово — `active (running)`. Зелёная точка `()` тоже означает, что всё хорошо.

Остановить сервис (например, если нужно внести изменения):

```
systemctl --user stop rag-watcher
```

Запустить сервис снова:

```
systemctl --user start rag-watcher
```

Следить за логами в реальном времени (флаг `-f` означает *follow*, т.е. показывать новые строки по мере их появления):

```
journalctl --user -u rag-watcher -f
```

[>>] Что должно произойти

Что увидите:

```
Jan 15 10:23:45 computer python[12345]: Watching directory: /home/username/KMS/archive
Jan 15 10:23:45 computer python[12345]: Ready. Waiting for new files...
```

Когда появится новый документ, здесь же в реальном времени будут показаны строки о его обработке. Чтобы выйти из режима просмотра логов, нажмите **Ctrl+C**.

[i] Совет

Команду `journalctl --user -u rag-watcher -n 50` (без флага `-f`) можно использовать для просмотра последних 50 строк лога, не «приликая» к нему. Это удобно для быстрой проверки, что происходило последнее время.

4.4 Проверка работы наблюдателя

Убедимся, что всё работает, на практике:

1. Убедитесь, что сервис запущен: `systemctl --user status rag-watcher`
2. Откройте второй терминал для мониторинга: `journalctl --user -u rag-watcher -f`
3. В первом терминале скопируйте новый документ в архив:

```
cp ~/Загрузки/test_article.pdf ~/KMS/archive/
```

4. Подождите 30-60 секунд и наблюдайте за вторым терминалом.

[>>] Что должно произойти

Что увидите в логах:

```
Detected new file: test_article.pdf
Running ingestion...
  Extracting text... done
  Creating embeddings... done
  Stored in ChromaDB.
Running doc_to_obsidian...
  Generating summary... done
  Created note: ~/KMS/notes/Articles/test_article.md
Done processing test_article.pdf
```

5. Откройте Obsidian и нажмите Ctrl+R для обновления — должна появиться новая заметка.

4.5 Работа с USB-дискком

[!] Важно

Если вы используете USB-диск как хранилище документов, рекомендуется **подключать его до запуска компьютера** или **перед входом в систему**. Это гарантирует, что диск будет примонтирован до того, как наблюдатель начнёт работу.

Если вы подключили USB после того, как компьютер уже работает:

Система автоматически примонтирует диск — обычно в `/run/media/$USER/ИМЯ_ДИСКА` (на части систем — в `/media/$USER/ИМЯ_ДИСКА`). Символическая ссылка `~/KMS/archive` «оживёт»

автоматически. В течение ~30 секунд наблюдатель перезапустит слежение **и выполнит догоняющее сканирование**: файлы, добавленные на диск пока он был отключён, будут автоматически проиндексированы и получают заметки. Дубликаты при этом не создаются — индексация сверяется по содержимому (хэш), а заметки — по полю `source_file`.

[i] Совет

наблюдатель следит за папкой в режиме опроса (а не через `inotify`) — именно поэтому он надёжно видит изменения на USB/NTFS и через символическую ссылку.

Проверьте, что ссылка работает:

```
ls ~/KMS/archive
```

[>>] Что должно произойти

Что увидите: Список файлов на вашем USB-диске. Если видите ошибку `No such file or directory` или `Too many levels of symbolic links` — USB не примонтирован. Попробуйте извлечь и снова подключить диск.

Если нужно перезапустить наблюдатель после подключения USB:

```
systemctl --user restart rag-watcher
```

[!] Важно

Если диск не монтируется (для NTFS — ошибка вида `wrong fs type, bad option, bad superblock`): том помечен как «грязный» (был извлечён некорректно или использовался в Windows). Почините флаг и переподключите диск:

```
lsblk # найдите раздел диска, например /dev/sdb2
sudo ntfsfix /dev/sdb2
```

После этого извлеките и снова вставьте диск — он смонтируется нормально.

[i] Совет

Автозапуск без входа в систему. По умолчанию пользовательский сервис работает, только пока вы вошли в графическую сессию. Чтобы наблюдатель стартовал при загрузке компьютера и продолжал работать после выхода из системы, включите «linger» (один раз):

```
loginctl enable-linger $USER
```

Раздел 5. Ежедневная работа

После первоначальной настройки система требует минимального обслуживания. В этом разделе описан типичный рабочий процесс.

5.1 Добавление новых документов

Когда вы находите новый материал, просто добавьте его в архив в любом поддерживаемом формате. Дальше система всё сделает сама (если наблюдатель запущен).

Поддерживаемые форматы: PDF, DOCX, DOC, TXT, MD, XLSX, CSV, PPTX, ODT

Способ 1: Через файловый менеджер

Откройте файловый менеджер, найдите файл, перетащите (или скопируйте/вставьте) его в папку ~/KMS/archive. Готово.

Способ 2: Через терминал

```
cp ~/Загрузки/new_article.pdf ~/KMS/archive/  
cp ~/Загрузки/report.docx ~/KMS/archive/  
cp ~/Загрузки/data.xlsx ~/KMS/archive/
```

Способ 3: Организация по подпапкам

Система поддерживает вложенные папки в архиве. Вы можете организовать документы по темам, типам или годам:

```
mkdir -p ~/KMS/archive/C02_storage  
mkdir -p ~/KMS/archive/Reports  
cp ~/Загрузки/co2_paper.pdf ~/KMS/archive/C02_storage/  
cp ~/Загрузки/quarterly.xlsx ~/KMS/archive/Reports/
```

Наблюдатель отслеживает все подпапки внутри ~/KMS/archive, рекурсивно.

[i] Совет

Файлы форматов, не входящих в список .pdf .docx .doc .txt .md .xlsx .csv .pptx .odt, будут молча проигнорированы — система не выдаст ошибку, просто пропустит такой файл.

После копирования файла дождитесь 1-3 минут и проверьте появление заметки в Obsidian (Ctrl+R для обновления файлового дерева).

[i] Совет

Если вы одновременно добавляете много документов (например, 20-30 штук), дайте системе время на обработку. Наблюдатель обрабатывает файлы последовательно, примерно по 1-3 минуты на файл. Следить за прогрессом можно через: `journalctl --user -u rag-watcher -f`

5.2 Работа с Obsidian

Просмотр заметки о документе

Откройте Obsidian и в левой панели найдите папку Articles. Кликните на любую заметку. Вы увидите структурированный документ примерно такого вида:

```
---
title: "Enhanced Oil Recovery Using CO2 Injection in Carbonate Reservoirs"
authors: ["Smith J.", "Brown A."]
year: 2023
doi: "10.1016/j.petro.2023.01.001"
tags: [auto/co2-injection, auto/eor, auto/carbonate-reservoir]
source: "article_1.pdf"
file_type: PDF
---

## Резюме

Статья исследует эффективность закачки CO2 для повышения нефтеотдачи в карбонатных коллекторах...
```

Ключевые выводы

- Закачка CO₂ повышает нефтеотдачу на 12-18% в условиях...
- Минеральная ловушка CO₂ наиболее эффективна при...

Связанные темы

[[CO2 Sequestration]] | [[Carbonate Reservoirs]] | [[EOR Methods]]

Граф связей

Нажмите Ctrl+G для открытия графа. Чем больше документов в вашем архиве, тем ценнее граф:

- **Крупные узлы** — заметки с множеством связей (важные темы или авторы).
- **Кластеры** — группы документов по смежным темам.
- **Мосты** — документы, соединяющие разные тематические кластеры (часто наиболее интересны для обзоров).

Поиск по тегам

Кликните на тег в любой заметке (например, #auto/co2-injection) — откроется список всех заметок с этим тегом. Это мощный инструмент для тематического поиска.

Теги с префиксом auto/

Все теги, созданные системой автоматически, имеют префикс auto/. Это позволяет легко отличить автоматически проставленные теги от тех, что вы добавили вручную. Вы можете добавлять свои теги прямо в заметках Obsidian, и они не будут перезаписаны при повторной обработке.

5.3 Вопросы через чат-интерфейс

Запустите чат:

```
cd ~/KMS/rag_system && source .venv/bin/activate && python chat_ui.py
```

Откройте `http://127.0.0.1:7860` в браузере.

Советы по формулировке вопросов:

Хорошие вопросы — конкретные и с контекстом:

- Плохо: «Что такое CO2?»
- Хорошо: «Какова роль CO2 в минеральной ловушке в карбонатных коллекторах по данным статей в моей базе?»
- Плохо: «Расскажи о нефти»
- Хорошо: «Какие методы повышения нефтеотдачи упоминаются в статьях 2020-2023 годов?»
- Хорошо: «Перечисли всех авторов, изучавших тему минеральных ловушек CO2»
- Хорошо: «Какие противоречия существуют между статьями о закачке воды и закачке CO2?»
- Хорошо: «Summarize the key findings about permeability in tight reservoirs»

Параметр top-k:

Если в интерфейсе есть ползунок top-k, он управляет количеством фрагментов базы знаний, которые система использует для формирования ответа:

- **top-k = 3** — быстрый ответ, основан на 3 наиболее релевантных фрагментах. Подходит для чётких вопросов.
- **top-k = 7-10** — более полный ответ, охватывает больше материала. Занимает больше времени. Подходит для обобщающих вопросов.

5.4 Вопросы из командной строки

Если вам не нужен браузер, можно задавать вопросы прямо из терминала:

```
python rag_engine.py "Какова роль кальцита в минеральной ловушке CO2?"
```

[>>] Что должно произойти

Что увидите:

Searching knowledge base...

Found 5 relevant chunks from 3 documents.

Generating answer...

Кальцит играет ключевую роль в процессе минеральной ловушки CO₂...

[далее развёрнутый ответ]

Sources:

- *article_1.pdf (pages 12-14)*
- *review_2023.pdf (pages 5-7)*
- *article_3.pdf (page 22)*

Получить статистику базы знаний:

```
python rag_engine.py --stats
```

[>>] Что должно произойти

Что увидите:

Knowledge Base Statistics

```
Total documents indexed: 47
Total chunks in ChromaDB: 14,832
Embedding model:         intfloat/multilingual-e5-large
LLM provider:            ollama
LLM model:               qwen3.5:latest
Last updated:            2026-01-15 14:23:11
Archive size:            2.3 GB (47 files)
ChromaDB size:           1.1 GB
```

5.5 Обновление таксономии тегов

Со временем в вашем архиве накопятся сотни автоматически созданных тегов. Полезно периодически проверять, какие теги встречаются чаще всего, и добавлять наиболее популярные в «основную» таксономию.

Посмотрим на самые частые автоматические теги:

```
grep -r "auto/" ~/KMS/notes/ | grep "tags:" | sort | uniq -c | sort -rn | head -20
```

Разберём эту команду по частям:

- `grep -r "auto/" ~/KMS/notes/` — рекурсивно ищем строки, содержащие `auto/`, во всех заметках.
- `| grep "tags:"` — из них оставляем только строки с тегами (в блоке метаданных).
- `| sort` — сортируем для подготовки к следующему шагу.
- `| uniq -c` — считаем количество одинаковых строк.

- | `sort -rn` — сортируем по убыванию числа (-r обратный, -n числовой порядок).
- | `head -20` — берём только первые 20 результатов.

[>>] Что должно произойти**Что увидите:**

```
23 tags: [auto/co2-injection, ...
19 tags: [auto/reservoir-simulation, ...
17 tags: [auto/eor, ...
15 tags: [auto/carbonate-reservoir, ...
12 tags: [auto/permeability, ...
...
```

Видите теги, которые встречаются более 10 раз? Их можно добавить в `config.py` как «базовые теги» — это улучшит качество автоматической классификации новых документов.

Откройте конфиг:

```
nano ~/KMS/rag_system/config.py
```

Найдите раздел `TAXONOMY` (Ctrl+W для поиска) и добавьте частые теги в список. Сохраните файл (Ctrl+X Y Enter).

Если хотите пересоздать все заметки с новыми тегами:

```
cd ~/KMS/rag_system
source .venv/bin/activate
python doc_to_obsidian.py --force
```

[!] Важно

Флаг `--force` пересоздаёт все заметки Obsidian (без него повторный запуск пропускает уже обработанные файлы — по полю `source_file`, так что дубликатов не возникает). Если вы вручную дописывали что-то в автоматические заметки, сохраните эти правки отдельно перед запуском с `--force` — он перезапишет содержимое.

Раздел 6. Устранение проблем

В этом разделе собраны наиболее часто встречающиеся проблемы и способы их решения. Если ваша ситуация не описана здесь — посмотрите в логах системы (`tail -f ~/KMS/rag_system/logs/rag_system.log`) подробности ошибки.

6.1 «command not found» при запуске python-скриптов

Симптом:

```
bash: python: command not found
```

или

```
ModuleNotFoundError: No module named 'chromadb'
```

Причина: Виртуальное окружение не активировано.

Решение:

```
source ~/KMS/rag_system/.venv/bin/activate
```

После этого в начале строки появится `(.venv)` и все команды заработают. Нужно делать это при **каждом открытии нового терминала**.

[i] Совет

Чтобы не забывать об активации, можно добавить псевдоним. Откройте `~/.bashrc` (конфигурация терминала) и добавьте в конец строку:

```
alias rag='cd ~/KMS/rag_system && source .venv/bin/activate'
```

Теперь достаточно набрать `rag` в любом терминале — и вы окажетесь в нужной папке с активированным окружением.

6.2 «Ollama недоступна» / «Connection refused»

Симптом:

```
Error: Could not connect to Ollama at http://localhost:11434
Connection refused
```

Причина: Сервис Ollama не запущен.

Решение:

```
ollama serve
```

[>>] Что должно произойти

Что увидите:

```
2024/01/15 10:23:45 routes.go:1007: Listening on 127.0.0.1:11434 (version 0.1.17)
```

Оставьте этот терминал открытым (или запустите `ollama serve &` с символом `&` для запуска в фоне) и повторите вашу команду.

[i] Совет

Обычно Ollama запускается автоматически при старте системы. Если это происходит регулярно, попробуйте:

```
sudo systemctl enable ollama
sudo systemctl start ollama
```

6.3 «Модель не найдена» / «Model not found»

Симптом:

```
Error: model 'mistral' not found
```

Решение:

Сначала посмотрим, какие модели есть:

```
ollama list
```

Если модель не в списке, скачаем её:

```
ollama pull mistral
```

Если список пустой, возможно, Ollama недавно переустановлена или данные были удалены. Скачайте нужную модель заново.

6.4 Ответ на русском плохого качества

Симптом: Модель отвечает на русский запрос очень кратко, смешивает языки или делает грубые грамматические ошибки.

Причина: Модель `mistral`, хотя и понимает русский, лучше работает с английским. Для лучшего качества на русском рекомендуются модели `qwen2.5:7b` или `llama3.1:8b`.

[!] Важно

Для запуска `llama3.1:8b` нужно минимум **16 ГБ оперативной памяти**. Проверьте: `free -h` — в строке `Mem`: смотрите столбец `total`.

Скачаем рекомендованную модель (файл ~5 ГБ):

```
ollama pull qwen2.5:7b
```

[i] Совет

Это нормально: Скачивание займёт 15-30 минут.

После скачивания обновим конфиг:

```
nano ~/KMS/rag_system/config.py
```

Найдите строки и измените на:

```
OLLAMA_MODEL: str = "qwen2.5:7b"  
LLM_PROVIDER: str = "ollama"  
LLM_MODEL: str = "qwen2.5:7b"
```

Сохраните (Ctrl+X Y Enter). Перезапустите чат-интерфейс или rag_engine — изменения применятся автоматически.

6.5 Документ не индексируется

Вариант А: Сканированный PDF

Симптом: После добавления PDF-файла система выводит:

```
Warning: article_name.pdf - No text extracted (0 characters)  
Skipping: file may be a scanned document.
```

Причина: Некоторые PDF — это просто сканы страниц (изображения), в них нет «настоящего» текста, только картинки. Стандартные инструменты извлечения текста не могут их обработать.

Как отличить: Попробуйте открыть PDF в любом просмотрщике и выделить текст мышью. Если не получается — документ сканирован.

Что делать:

Нужно OCR (*Optical Character Recognition* — оптическое распознавание символов) — преобразование изображений текста в настоящий текст. Установите утилиту ocrmypdf:

```
sudo apt install ocrmypdf tesseract-ocr tesseract-ocr-rus -y
```

Обработайте файл (создаст новый PDF с текстовым слоем):

```
ocrmypdf -l rus+eng ~/KMS/archive/scanned_article.pdf  
~/KMS/archive/scanned_article_ocr.pdf
```

Теперь можно проиндексировать обработанный файл:

```
cd ~/KMS/rag_system && source .venv/bin/activate
python ingest.py          # переиндексация архива: новый OCR-файл добавится, остальное
                           пропустится (дедуп по хэшу)
python doc_to_obsidian.py  # заметка для нового файла (уже обработанные пропускаются по
                           source_file)
```

[i] Совет

Флаг `-l rus+eng` указывает OCR использовать оба языка. Если документ только на английском, используйте `-l eng` — обработка будет точнее и быстрее.

Вариант Б: Файл DOC не распознаётся**Симптом:**

```
Warning: document.doc - extraction failed, text empty
```

Причина: Для старого формата `.doc` система использует LibreOffice. Если LibreOffice не установлен — обработка не пройдёт.

Решение:

```
sudo apt install libreoffice antiword -y
# Перезапустить наблюдатель:
systemctl --user restart rag-watcher
```

После этого скопируйте файл заново (или наблюдатель сам подхватит его при перезапуске).

Вариант В: Неподдерживаемый формат

Файлы с расширениями не из списка `.pdf .docx .doc .txt .md .xlsx .csv .pptx .odt` молча игнорируются — это нормальное поведение, не ошибка.

6.6 Наблюдатель не видит новые файлы

Шаг 1: Проверим статус сервиса:

```
systemctl --user status rag-watcher
```

Если видите `inactive (dead)` или `failed` — сервис не работает.

Шаг 2: Посмотрим последние строки лога:

```
journalctl --user -u rag-watcher -n 50
```

Прочитайте последние строки — там должна быть информация об ошибке.

Шаг 3: Перезапустим сервис:

```
systemctl --user restart rag-watcher
```

Если ошибка повторяется:

Частая причина — виртуальное окружение Python не найдено. Проверьте, что путь в файле сервиса правильный:

```
cat ~/.config/systemd/user/rag-watcher.service
```

В строке ExecStart= должен быть полный путь к Python в виртуальном окружении вида:

```
ExecStart=/home/username/KMS/rag_system/.venv/bin/python watcher.py
```

Если имя пользователя отличается от username — отредактируйте файл сервиса, затем:

```
systemctl --user daemon-reload
systemctl --user restart rag-watcher
```

6.7 Нет места на диске

Симптом: Ошибки вида No space left on device или система работает медленно.

Диагностика:

Посмотрим, сколько вообще места на дисках:

```
df -h
```

[>>] Что должно произойти**Что увидите:**

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda2	234G	187G	35G	85%	/
/dev/sdb1	15G	8.2G	6.8G	55%	/media/username/MY_USB

Обратите внимание на столбец Use%. Если более 90% — нужно освобождать место.

Посмотрим, сколько занимает база ChromaDB:

```
du -sh ~/KMS/rag_system/chroma_db/
```

[>>] Что должно произойти**Что увидите:**

```
2.3G    /home/username/KMS/rag_system/chroma_db/
```

База ChromaDB растёт по мере добавления документов. Примерно: 1 ГБ исходных документов 0.5 ГБ ChromaDB.

Варианты решения:

1. **Освободить место:** Удалите ненужные файлы из Загрузок и других папок.
2. **Переместить ChromaDB на другой диск:** В config.py параметр базы называется CHROMA_PERSIST_DIR (по умолчанию — папка chroma_db внутри rag_system). Чтобы перенести базу, замените строку CHROMA_PERSIST_DIR = BASE_DIR / "chroma_db" на абсолютный путь, например CHROMA_PERSIST_DIR = Path("/mnt/big_disk/chroma_db").
3. **Очистить кеш pip:** pip cache purge (освободит 100-500 МБ).
4. **Очистить старые журналы:** journalctl --user --vacuum-size=100M (оставит только последние 100 МБ логов).

6.8 Забыл, как запустить чат

Это самый частый вопрос! Вот полная команда, которую можно скопировать прямо из этого руководства:

```
cd ~/KMS/rag_system && source .venv/bin/activate && python chat_ui.py
```

Три команды объединены через && (запустить следующую, только если предыдущая успешна). После запуска откройте браузер и перейдите на <http://127.0.0.1:7860>.

[i] Совет

Добавьте эту команду как псевдоним в ~/.bashrc:

```
echo "alias ragchat='cd ~/KMS/rag_system && source .venv/bin/activate && python  
chat_ui.py'" >> ~/.bashrc  
source ~/.bashrc
```

После этого достаточно вводить ragchat в любом терминале.

6.9 Ollama работает очень медленно

Симптом: Ответ на простой вопрос занимает более 2-3 минут.

Причина: Система генерирует ответы на CPU (процессор), а не на GPU (видеокарта). Для языковых моделей GPU даёт ускорение в 5-20 раз.

Проверим, использует ли Ollama GPU:

```
ollama ps
```

[>>] Что должно произойти

Что увидите:

NAME	ID	SIZE	PROCESSOR	UNTIL
mistral:latest	f974a74358d6	5.4 GB	100% GPU	4 minutes from now

Если в столбце PROCESSOR написано 100% CPU или 0% GPU — модель работает без GPU.

Если у вас NVIDIA GPU:

Установите CUDA следуя официальной документации NVIDIA для вашей версии Ubuntu. После установки CUDA Ollama автоматически начнёт использовать GPU.

Если GPU нет или это встроенная Intel/AMD:

К сожалению, значительного ускорения добиться не получится. Рассмотрите два варианта:

1. Использовать более лёгкую модель:

```
ollama pull mistral:7b-instruct-q4_0
```

В конфиге укажите: OLLAMA_MODEL: str = "mistral:7b-instruct-q4_0" — это более «сжатая» версия модели, работает быстрее, но чуть хуже по качеству.

2. **Переключиться на облачный провайдер Groq** — он бесплатный и работает значительно быстрее. Подробнее — в Разделе 7.

6.10 Ошибка при установке зависимостей

Симптом:

```
error: legacy-install-failure
× Encountered error while trying to install package.
└─> chroma-hnswlib
```

Причина: Для вашей версии Python нет готового пакета, и chroma-hnswlib пытается собраться из исходников. **Самая частая причина — слишком новый Python (3.13+):** под него готовых пакетов ChromaDB ещё нет.

Решение:

Если у вас Python **3.13 или новее** — пересоздайте окружение на Python 3.12 (см. также раздел 1.3):

```
sudo apt install python3.12 python3.12-venv -y
cd ~/KMS/rag_system && rm -rf .venv && python3.12 -m venv .venv
source .venv/bin/activate && pip install -r requirements.txt
```

Если Python 3.10–3.12, но не хватает инструментов сборки:

```
sudo apt install python3-dev build-essential cmake -y
pip install -r requirements.txt
```

6.11 Ускорение индексации через видеокарту (GPU / NVIDIA CUDA)

По умолчанию система ставит CPU-сборку PyTorch — она работает везде, но создание эмбедингов идёт медленно (особенно с моделью e5-large). Если у вас есть видеокарта NVIDIA (например, GeForce RTX 30xx/40xx), можно поставить CUDA-сборку PyTorch и ускорить индексацию **примерно в 20–25 раз**.

Шаг 1. Проверьте видеокарту и драйвер:

```
nvidia-smi
```

Вы увидите модель карты и строку CUDA Version: 12.x (или новее). Если команды нет — установите проприетарный драйвер NVIDIA через «Программы и обновления» «Дополнительные драйверы».

Шаг 2. Замените torch на CUDA-сборку (в активированном окружении):

```
cd ~/KMS/rag_system && source .venv/bin/activate  
pip install --force-reinstall --index-url https://download.pytorch.org/whl/cu128 torch
```

[i] Совет

cu128 — это CUDA 12.8; подходит для современных драйверов. Загрузка большая (~3 ГБ: сам torch + библиотеки cuDNN/cuBLAS). Если используете uv: `uv pip install --reinstall-package torch --index-url https://download.pytorch.org/whl/cu128 torch`.

Шаг 3. Проверьте, что GPU виден:

```
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

Должно вывести True и название вашей карты.

После этого `python ingest.py` и наблюдатель автоматически используют GPU — менять настройки не нужно (sentence-transformers сам выбирает CUDA).

[!] Важно

Про видеопамять (VRAM). Эмбединги и языковая модель Ollama делят одну видеопамять. На картах с **8 ГБ** их совместная работа возможна, но «впритык». Поэтому чат-интерфейс (`chat_ui.py`) намеренно считает эмбединги **на CPU** (встраивание одного короткого вопроса и так мгновенно), оставляя всю видеопамять под LLM — так ответы генерируются быстрее. Пакетная индексация при этом по-прежнему идёт на GPU. Если хотите задействовать GPU и в чате — запускайте `CUDA_VISIBLE_DEVICES=0 python chat_ui.py`.

[i] Совет

на GPU полная переиндексация (`python ingest.py --reset`) проходит в десятки раз быстрее, чем на CPU (в нашем тесте на RTX 3070 — около 26×).

Раздел 7. Управление LLM-провайдерами

RAG-система версии 5.0 (v4.2) поддерживает **5 провайдеров** через единый интерфейс — файл `llm_provider.py`. Это означает, что вы можете переключаться между локальной Ollama, облачными Groq и DeepSeek, агрегатором OpenRouter и локальным LM Studio — всё через три строки в `config.py`, не меняя остального кода.

Почему это важно?

- Нет интернета или нужна полная конфиденциальность **ollama** (локально, бесплатно)
- Нужны мощные модели без GPU **groq** (облако, бесплатно, до 14 400 запросов/день)
- Нужны самые умные рассуждения **deepseek** (облако, дёшево, \$0.07/1M токенов)
- Хотите выбирать из 50+ моделей **openrouter** (есть бесплатные варианты)
- Хотите красивый GUI + локальность **lmstudio** (локально, бесплатно)

[i] Совет

«Размышляющие» модели (**qwen3, deepseek-r1 и подобные**). Они сначала генерируют скрытую «цепочку рассуждений». Система **автоматически отключает** этот режим (`think: false`) при обращении к Ollama — иначе модель тратит весь лимит токенов на «размышления» и возвращает пустой ответ. Если после переключения на ollama-модель ответы вдруг стали пустыми («Ollama вернула пустой ответ») — это почти наверняка «думающая» модель; убедитесь, что у вас актуальные `llm_provider.py` и `doc_to_obsidian.py` (в них эта правка уже есть).

Таблица провайдеров

Провайдер	Описание	API-ключ	Бесплатно
ollama	Локальная Ollama (уже установлена)	Не нужен	Да
groq	Groq Cloud — быстрые LLM в облаке	console.groq.com/keys	14 400 req/день
deepseek	DeepSeek API	platform.deepseek.com	Платно (\$0.07/1M)
openrouter	50+ моделей от разных провайдеров	openrouter.ai/settings/keys	Есть бесплатные
lmstudio	Локальный LM Studio (GUI)	Не нужен	Да

7.1 Переключение провайдера

Откройте файл конфигурации:

```
nano ~/KMS/rag_system/config.py
```

Найдите и отредактируйте **три строки** (Ctrl+W LLM_PROVIDER):

```
LLM_PROVIDER: str = "ollama"  
LLM_MODEL:    str = "qwen2.5:7b"  
LLM_API_KEY:  str = ""
```

Сохраните (Ctrl+X Y Enter) и перезапустите чат или скрипт.

Примеры настройки для каждого провайдера

Ollama (локально, по умолчанию):

```
LLM_PROVIDER: str = "ollama"  
LLM_MODEL:    str = "qwen2.5:7b"  
LLM_API_KEY:  str = ""
```

[i] Совет

Перед использованием убедитесь, что модель скачана: `ollama pull qwen2.5:7b`

Groq (облако, бесплатно):

```
LLM_PROVIDER: str = "groq"  
LLM_MODEL:    str = "llama-3.3-70b-versatile"  
LLM_API_KEY:  str = "gsk_ВАШ_КЛЮЧ_ЗДЕСЬ"
```

Получить бесплатный API-ключ: console.groq.com/keys

[i] Groq — это облачный сервис, который предоставляет мощные модели с очень высокой скоростью генерации. Лимит бесплатного тарифа: 14 400 запросов в день. Данные отправляются на серверы Groq (США).

DeepSeek (облако, платно):

```
LLM_PROVIDER: str = "deepseek"  
LLM_MODEL:    str = "deepseek-chat"  
LLM_API_KEY:  str = "sk-ВАШ_КЛЮЧ_ЗДЕСЬ"
```

Получить ключ и пополнить баланс: platform.deepseek.com

[i] DeepSeek — китайский провайдер с очень низкими ценами (\$0.07 за 1 миллион токенов для deepseek-chat). Отлично справляется с техническими текстами. Для задач, требующих глубокого рассуждения, используйте модель deepseek-reasoner.

OpenRouter (50+ моделей, есть бесплатные):

```
LLM_PROVIDER: str = "openrouter"
LLM_MODEL:     str = "qwen/qwen3-14b:free"
LLM_API_KEY:   str = "sk-or-ВАШ_КЛЮЧ_ЗДЕСЬ"
```

Получить ключ: openrouter.ai/settings/keys

[i] OpenRouter — агрегатор, который даёт доступ к десяткам моделей через единый API. Модели с суффиксом :free (например, qwen/qwen3-14b:free, deepseek/deepseek-r1:free) доступны бесплатно с определёнными ограничениями по скорости.

LM Studio (локально, с GUI):

```
LLM_PROVIDER: str = "lmstudio"
LLM_MODEL:     str = "qwen2.5-7b-instruct"
LLM_API_KEY:   str = ""
```

[i] Совет

LM Studio — это приложение с графическим интерфейсом для запуска локальных LLM. Перед использованием: 1) Скачайте LM Studio с lmstudio.ai, 2) Загрузите нужную модель через интерфейс, 3) Запустите локальный сервер в LM Studio (меню Local Server). RAG-система будет подключаться к нему автоматически по адресу <http://localhost:1234>.

7.2 Рекомендуемые модели для нефтегазовой тематики

При работе с технической, нормативной и научной документацией по нефтегазовой отрасли рекомендуются следующие модели:

Ollama (локально, без интернета)

Модель	Размер	ОЗУ	Качество русского	Рекомендация
qwen2.5:7b	~5 ГБ	8 ГБ		Рекомендуется — лучший баланс качества и скорости
qwen2.5:14b	~9 ГБ	16 ГБ		Лучше при анализе длинных документов
deepseek-r1:7b	~5 ГБ	8 ГБ		Хорош для технических рассуждений
mistral	~4 ГБ	8 ГБ		Базовый вариант, подходит для начала

Модель	Размер	ОЗУ	Качество русского	Рекомендация
llama3.1:8b	~5 ГБ	16 ГБ		Хороший русский, особенно для диалогов

Загрузка модели:

```
ollama pull qwen2.5:7b
```

Groq (облако, бесплатно)

Модель	Описание
llama-3.3-70b-versatile	Мощная модель для комплексного анализа
deepseek-r1-distill-llama-70b	Лучшая для технических рассуждений
mixtral-8x7b-32768	Хорошо справляется с длинными текстами

[i] Совет

Groq особенно хорош, если у вас нет мощного GPU, но нужны ответы от большой модели.

OpenRouter (бесплатные варианты)

Модель	Описание
qwen/qwen3-14b:free	Отличный русский, бесплатно
deepseek/deepseek-r1:free	Сильное техническое рассуждение, бесплатно
meta-llama/llama-3.3-70b-instruct:free	Мощная открытая модель, бесплатно

DeepSeek (платно, низкие цены)

Модель	Применение
deepseek-chat	Стандартные вопросы по документам
deepseek-reasoner	Сложный анализ, сравнение, рассуждения

7.3 Переменные окружения

Хранить API-ключи прямо в config.py — не лучшая практика (файл может попасть в резервную копию или систему контроля версий). Альтернатива — переменные окружения.

Добавьте ключи в файл ~/.bashrc (конфигурация вашего терминала):

```
nano ~/.bashrc
```

Добавьте в конец файла:

```
# API-ключи для RAG KMS
export RAG_GROQ_API_KEY="gsk_ваш_groq_ключ"
export RAG_DEEPSEEK_API_KEY="sk-ваш_deepseek_ключ"
export RAG_OPENROUTER_API_KEY="sk-or-ваш_openrouter_ключ"
```

Сохраните (Ctrl+X Y Enter) и активируйте:

```
source ~/.bashrc
```

Если переменные окружения заданы, `llm_provider.py` автоматически подберёт нужный ключ — поле `LLM_API_KEY` в `config.py` можно оставить пустым:

```
LLM_PROVIDER: str = "groq"
LLM_MODEL:    str = "llama-3.3-70b-versatile"
LLM_API_KEY:  str = "" # ← берётся из переменной RAG_GROQ_API_KEY
```

[!] Важно

Никому не сообщайте API-ключи. Не добавляйте файл `~/.bashrc` с ключами в резервные копии, которые хранятся в облаке.

7.4 Устранение проблем с провайдерами

Ошибка 401 (Unauthorized — неверный ключ)

Симптом:

```
Error: 401 Unauthorized
APIError: Invalid API key
```

Что делать:

1. Проверьте, что ключ скопирован полностью (без лишних пробелов)
2. Убедитесь, что ключ активен в личном кабинете провайдера
3. Проверьте, что в `config.py` правильно указан `LLM_PROVIDER`

Быстрая проверка ключа Groq:

```
curl -s -H "Authorization: Bearer $RAG_GROQ_API_KEY" \
https://api.groq.com/openai/v1/models | python3 -m json.tool | head -20
```

Ошибка 429 (Rate Limit — превышен лимит запросов)

Симптом:

```
Error: 429 Too Many Requests
RateLimitError: Rate limit exceeded
```

Что делать:

- Для **Groq**: бесплатный лимит — 14 400 запросов/день. Дождитесь сброса (в полночь UTC) или переключитесь временно на ollama.
- Для **OpenRouter** с бесплатными моделями: добавьте задержку или переключитесь на платную квоту.
- Для **DeepSeek**: пополните баланс на платформе.

Временное переключение на Ollama без изменения config.py:

```
# Запустить скрипт с другим провайдером (если поддерживается):
LLM_PROVIDER=ollama python rag_engine.py "ваш вопрос"
```

Timeout (провайдер не отвечает)**Симптом:**

```
Error: ConnectionTimeout
ReadTimeout: The read operation timed out
```

Что делать:

1. Проверьте интернет-соединение: `ping api.groq.com`
2. Переключитесь на локальный провайдер (ollama или lmstudio)
3. Попробуйте позже — возможны временные проблемы на стороне провайдера

LM Studio не отвечает**Симптом:**

```
Error: Could not connect to LM Studio at http://localhost:1234
```

Что делать:

1. Убедитесь, что LM Studio запущен
2. В LM Studio перейдите на вкладку «Local Server»
3. Нажмите кнопку «Start Server»
4. Убедитесь, что модель загружена (зелёный индикатор)

Раздел 8. Обновление Ollama и LLM

С течением времени выходят новые версии Ollama с улучшениями производительности, а также новые языковые модели с лучшим качеством. В этом разделе описано, как безопасно обновить всё.

Обновление Ollama

Для обновления Ollama до последней версии используется та же команда, что и при первоначальной установке:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Скрипт автоматически обнаружит установленную версию и обновит её, не затрагивая скачанные модели.

Проверьте версию после обновления:

```
ollama --version
```

[>>] Что должно произойти
Что увидите:

```
ollama version is 0.3.12
```

Просмотр установленных моделей

```
ollama list
```

[>>] Что должно произойти
Что увидите:

NAME	ID	SIZE	MODIFIED
qwen2.5:7b	a04eb7c1...	4.7 GB	2 weeks ago
mistral:latest	f974a743...	4.1 GB	1 month ago
nomic-embed-text:latest	0a109f42...	274 MB	3 weeks ago

Загрузка новой модели

```
ollama pull qwen2.5:14b
```

Модель будет скачана и добавлена к уже имеющимся. После скачивания обновите config.py:

```
nano ~/KMS/rag_system/config.py
```

Замените значения:

```
OLLAMA_MODEL: str = "qwen2.5:14b"
LLM_MODEL:    str = "qwen2.5:14b"
```

Обновление существующей модели

Повторный запуск `ollama pull` обновляет модель до последней версии:

```
ollama pull qwen2.5:7b
```

Если последняя версия уже установлена, вы увидите:

```
pulling manifest
Up to date.
```

Удаление старой модели

Чтобы освободить место на диске, можно удалить модели, которые больше не используются:

```
ollama rm mistral
```

[>>] Что должно произойти
Что увидите:

```
deleted 'mistral'
```

[!] **Важно**
Перед удалением убедитесь, что в `config.py` указана другая модель в качестве основной.

Управление моделями — сводная таблица

Действие	Команда
Обновить Ollama	<code>curl -fsSL https://ollama.com/install.sh sh</code>
Проверить версию Ollama	<code>ollama --version</code>
Список установленных моделей	<code>ollama list</code>

Действие	Команда
Загрузить новую модель	<code>ollama pull qwen2.5:14b</code>
Обновить существующую модель	<code>ollama pull qwen2.5:7b</code> (повторный pull)
Удалить модель	<code>ollama rm mistral</code>
Статус запущенных моделей	<code>ollama ps</code>
Запустить сервер Ollama	<code>ollama serve</code>
Проверить API Ollama	<code>curl http://localhost:11434/api/tags</code>

Раздел 9. Подключение Claude Code через MCP (опционально)

Этот раздел опционален. Он описывает подключение RAG-системы к Claude Code — AI-ассистенту Anthropic, работающему в терминале. Если вы используете только веб-интерфейс (chat_ui.py), этот раздел можно пропустить. Интеграция с Claude Code даёт дополнительные возможности: Claude может самостоятельно решать, когда обращаться к вашей базе знаний, комбинировать результаты поиска с собственными знаниями и работать в контексте конкретного проекта.

9.0 Что такое Claude Code и MCP

Claude Code — AI-ассистент прямо в терминале

Claude Code — это инструмент от компании Anthropic, который позволяет общаться с AI-моделью Claude прямо из командной строки (терминала). Вы просто пишете вопрос на русском или английском языке, и Claude отвечает — как опытный коллега, который всё знает и всегда под рукой.

В отличие от веб-интерфейса, Claude Code умеет:

- Читать файлы на вашем компьютере
- Выполнять команды в терминале
- Вызывать внешние инструменты (в том числе вашу RAG-систему)
- Работать в контексте конкретного проекта

MCP — «розетка» для подключения инструментов

MCP (Model Context Protocol) — это открытый протокол, разработанный Anthropic, который позволяет Claude Code подключаться к внешним инструментам и источникам данных. Представьте его как стандартную «розетку»: любой инструмент, написанный по стандарту MCP, можно «воткнуть» в Claude Code, и он сразу начнёт им пользоваться.

Простая аналогия: если Claude Code — это ваш умный помощник, то MCP — это телефон, по которому он может позвонить в библиотеку (вашу RAG-систему) и спросить нужную информацию.

Что даёт интеграция RAG + Claude Code

Без интеграции:

- Claude отвечает только из своих общих знаний
- Не знает о ваших конкретных документах, регламентах, отчётах
- Может «галлюцинировать» технические детали

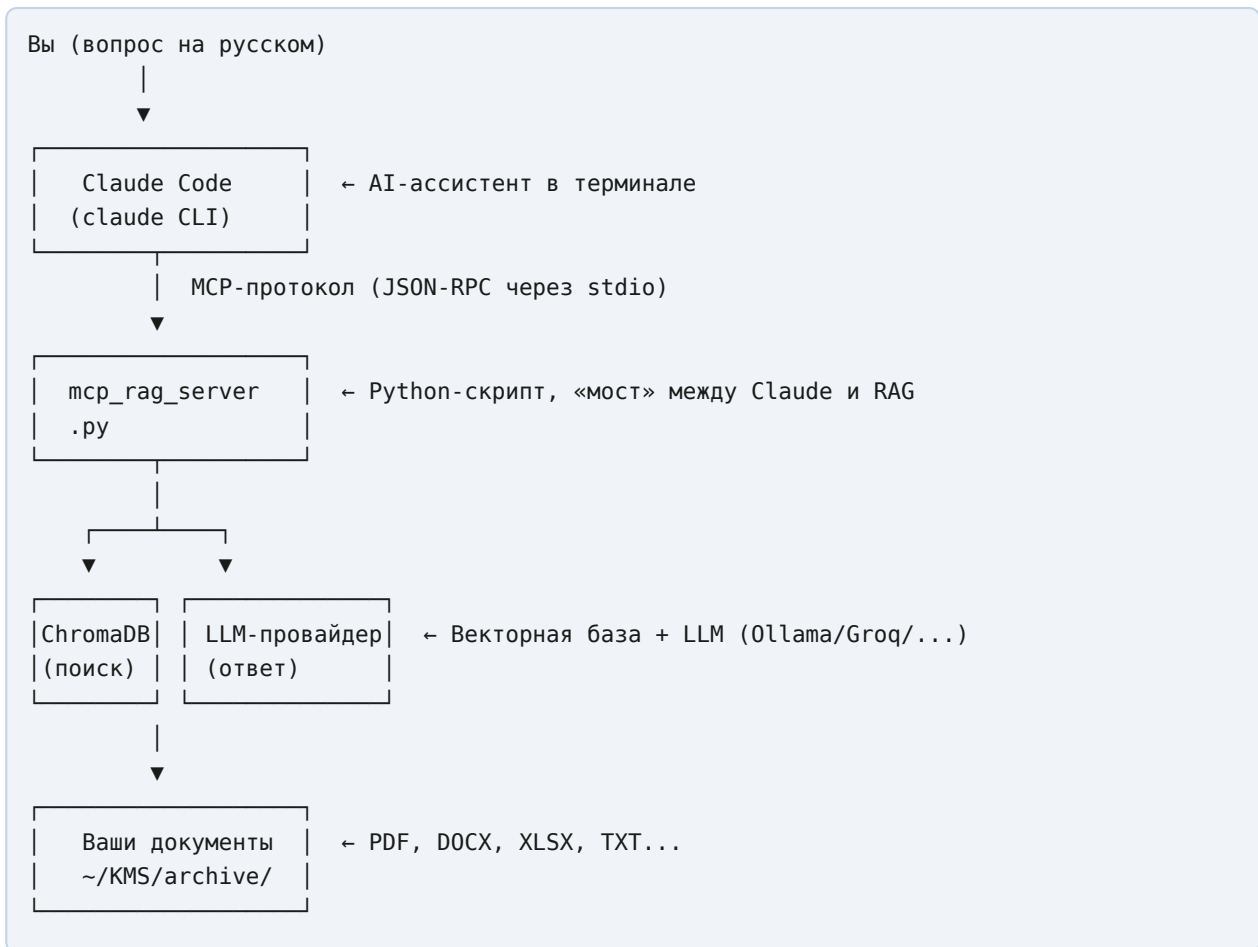
С интеграцией RAG:

- Claude ищет ответы в **ваших** документах: PDF, DOCX, XLSX и других
- Цитирует конкретные источники из архива
- Отвечает с учётом ваших регламентов, стандартов и отчётов
- Можно спрашивать на русском языке по нефтегазовой тематике

Таблица: 4 инструмента MCP-сервера

Инструмент	Что делает	Когда использовать
search_knowledge_base	Семантический поиск по документам + генерация ответа через LLM	Основной инструмент: вопросы по содержанию документов
list_documents	Показывает список всех проиндексированных файлов	Когда хотите знать, что вообще есть в базе
get_document_info	Подробная информация о конкретном файле	Когда нужны детали: дата, размер, количество чанков
get_stats	Статистика базы знаний и статус LLM-провайдера	Диагностика: всё ли работает

Схема работы системы



Как это работает по шагам:

1. Вы задаёте вопрос Claude Code в терминале
2. Claude понимает, что нужно обратиться к базе знаний, и вызывает `search_knowledge_base`
3. MCP-сервер ищет релевантные фрагменты в ChromaDB
4. Найденные фрагменты отправляются в LLM-провайдер для генерации ответа
5. Готовый ответ с цитатами возвращается вам через Claude Code

9.1 Установка Claude Code

Шаг 9.1.1: Проверить наличие Node.js

Claude Code — это npm-пакет, поэтому для его работы нужен Node.js (версия 18 или новее).

Откройте терминал и выполните:

```
node --version
```

[>>] Что вы увидите, если Node.js установлен:

```
v20.11.0
```

Если команда не найдена (`command not found`), установите Node.js:

```
# Для Ubuntu/Debian:
sudo apt update
sudo apt install nodejs npm -y

# Проверка после установки:
node --version
npm --version
```

[>>] Что должно получиться:

```
v20.11.0
10.2.3
```

Примечание: Номера версий могут отличаться — это нормально. Главное, чтобы Node.js был версии 18 или выше.

Шаг 9.1.2: Установить Claude Code через npm

Выполните в терминале:

```
npm install -g @anthropic-ai/claude-code
```

Флаг -g означает «глобальная установка» — Claude Code будет доступен из любой папки.

[>>] Что вы увидите в процессе установки:

```
added 247 packages in 15s
found 0 vulnerabilities
```

Если появляется ошибка EACCES (нет прав): Это означает, что npm-папка принадлежит root. Исправьте командой:

```
sudo npm install -g @anthropic-ai/claude-code
```

Шаг 9.1.3: Авторизация

Claude Code требует аутентификации через Anthropic. Есть два способа:

Способ А: Интерактивный вход (рекомендуется)

```
claude login
```

В браузере откроется страница авторизации Anthropic. Войдите в свой аккаунт или создайте новый на console.anthropic.com.

Способ Б: API-ключ (если нет браузера или нужна автоматизация)

1. Получите API-ключ на console.anthropic.com/settings/api-keys

2. Добавьте его в переменные окружения:

```
# Добавить в ~/.bashrc для постоянного сохранения:
echo 'export ANTHROPIC_API_KEY="sk-ant-ваш-ключ-здесь"' >> ~/.bashrc
source ~/.bashrc
```

[!] Важно

Не передавайте API-ключ другим людям. Это ваш личный ключ доступа.

Шаг 9.1.4: Проверка установки

```
claude --version
```

[>>] Что должно получиться:

```
Claude Code 1.x.x
```

Дополнительная проверка — попробуйте задать простой вопрос:

```
claude "Привет! Сколько будет 2+2?"
```

[>>] Ожидаемый ответ:

```
Привет! 2+2 = 4.
```

Если всё работает — Claude Code успешно установлен. Переходите к следующему разделу.

9.2 Подготовка MСР-сервера

Шаг 9.2.1: Убедиться, что RAG-система v5.0 установлена

Проверим, что все необходимые компоненты на месте:

```
# Проверить структуру папки KMS:
ls ~/KMS/
# Ожидаемый вывод:
# archive/  notes/   rag_system/

# Проверить содержимое rag_system:
ls ~/KMS/rag_system/

# Проверить, запущена ли Ollama (если используете Ollama):
ollama list
```

Проверим, активно ли виртуальное окружение:

```
source ~/KMS/rag_system/.venv/bin/activate
python --version
```

[>>] Ожидаемый вывод:

```
Python 3.11.x
```

Шаг 9.2.2: Проверить наличие файла mcp_rag_server.py

```
ls -la ~/KMS/rag_system/mcp_rag_server.py
```

[>>] Что должно получиться:

```
-rw-r--r-- 1 USERNAME USERNAME 8192 Jan 15 14:30 /home/USERNAME/KMS/rag_system/mcp_rag_server.py
```

Если файл есть — отлично, он уже создан в рамках установки RAG-системы v5.0. Если файл не найден — обратитесь к документации по установке RAG-системы.

Шаг 9.2.3: Тестовый запуск сервера вручную

Запустим сервер вручную, чтобы убедиться в отсутствии ошибок:

```
# Активируем окружение и запустим сервер:
source ~/KMS/rag_system/.venv/bin/activate
cd ~/KMS/rag_system/
python mcp_rag_server.py
```

[>>] Что вы увидите (диагностические сообщения):

```
[INFO] RAG MCP Server starting...
[INFO] Loading embedding model...
[INFO] ChromaDB connected: /home/USERNAME/KMS/rag_system/chroma_db
[INFO] Documents in index: 47
[INFO] MCP server ready. Waiting for connections...
```

[!] Важно

После этого сервер будет ждать входящих сообщений. Он не «зависает» — это нормальное поведение MCP-сервера. Остановите его нажатием **Ctrl+C**.

Примечание о stderr/stdout: MCP-сервер специально пишет все сообщения о состоянии (логи) в stderr, а stdout зарезервирован исключительно для передачи данных по MCP-протоколу. Это важно — если в stdout попадут посторонние сообщения, Claude Code не сможет распарсить ответы.

Шаг 9.2.4: Найти путь к Python-интерпретатору

Для настройки конфигурационного файла нам нужен **полный путь** к Python.

```
source ~/KMS/rag_system/.venv/bin/activate  
which python
```

[>>] Пример вывода:

```
/home/USERNAME/KMS/rag_system/.venv/bin/python
```

Узнать ваш USERNAME:

```
echo $USER
```

[>>] Пример вывода:

```
ivanov_ap
```

Запишите эти пути — они понадобятся в следующем разделе.

9.3 Подключение MCP к Claude Code

Шаг 9.3.1: Создать директорию ~/.claude/

```
mkdir -p ~/.claude
```

Флаг `-p` означает «создать, даже если папка уже существует» — команда безопасна для повторного запуска.

Проверка:

```
ls -la ~/ | grep .claude
```

[>>] Что должно получиться:

```
drwxr-xr-x 2 USERNAME USERNAME 4096 Jan 15 15:00 .claude
```

Шаг 9.3.2: Создать файл mcp_servers.json

Создадим конфигурационный файл. Сначала уточните ваш путь к Python (из шага 9.2.4).

```
# Узнаем USERNAME ещё раз:
echo $USER
# Например: ivanov_ap

# Узнаем путь к Python:
source ~/KMS/rag_system/.venv/bin/activate && which python
# Например: /home/ivanov_ap/KMS/rag_system/.venv/bin/python
```

Теперь создайте файл конфигурации, **заменяв USERNAME на ваш реальный логин**:

```
nano ~/.claude/mcp_servers.json
```

Вставьте следующее содержимое (замените USERNAME на ваш логин, полученный командой `echo $USER`):

```
{
  "rag_kms": {
    "command": "/home/USERNAME/KMS/rag_system/.venv/bin/python",
    "args": ["/home/USERNAME/KMS/rag_system/mcp_rag_server.py"],
    "cwd": "/home/USERNAME/KMS/rag_system"
  }
}
```

Пример для пользователя ivanov_ap:

```
{
  "rag_kms": {
    "command": "/home/ivanov_ap/KMS/rag_system/.venv/bin/python",
    "args": ["/home/ivanov_ap/KMS/rag_system/mcp_rag_server.py"],
    "cwd": "/home/ivanov_ap/KMS/rag_system"
  }
}
```

Сохраните файл: Ctrl+O, затем Enter, затем Ctrl+X.

Шаг 9.3.3: Проверить синтаксис JSON

JSON очень чувствителен к синтаксису: одна лишняя запятая или кавычка — и файл не читается. Проверим:

```
cat ~/.claude/mcp_servers.json | python3 -m json.tool
```

[>>] **Что должно получиться (файл корректен):**

```
{
  "rag_kms": {
    "command": "/home/ivanov_ap/KMS/rag_system/.venv/bin/python",
    "args": [
      "/home/ivanov_ap/KMS/rag_system/mcp_rag_server.py"
    ],
    "cwd": "/home/ivanov_ap/KMS/rag_system"
  }
}
```

Если есть ошибка:

```
json.decoder.JSONDecodeError: Expecting ',' delimiter: line 4 column 5 (char 87)
```

Откройте файл снова (`nano ~/.claude/mcp_servers.json`) и проверьте запятые и кавычки.

Шаг 9.3.4: Запустить Claude Code и проверить инструменты

```
cd ~/KMS
claude
```

После запуска введите команду:

```
/tools
```

[>>] Ожидаемый вывод — вы должны увидеть 4 инструмента из RAG:

```
Available tools:
...
rag_kms:search_knowledge_base - Семантический поиск по базе знаний с генерацией ответа
rag_kms:list_documents        - Список всех проиндексированных документов
rag_kms:get_document_info     - Подробная информация о конкретном документе
rag_kms:get_stats             - Статистика базы знаний и статус LLM-провайдера
...
```

Если инструменты появились — интеграция настроена успешно! Переходите к следующему разделу.

Не видите инструменты? Перейдите к разделу 9.7 — там описано решение этой проблемы.

9.4 Первые запросы

Запустите Claude Code из папки с проектом:

```
cd ~/KMS
claude
```

Теперь просто пишите вопросы на естественном языке. Claude Code сам решит, когда нужно обратиться к вашей базе знаний.

Примеры запросов по нефтегазовой тематике

Запрос 1: Поиск по регламентам

Какие требования к опрессовке нефтепровода указаны в наших регламентах?

[>>] Что произойдёт:

Claude Code вызывает: `search_knowledge_base("требования опрессовка нефтепровод регламент")`

Ответ:

Согласно документу "Регламент_ТО_нефтепровод_2024.pdf" (страница 12):
Опрессовка нефтепровода должна проводиться при давлении 1.25 от рабочего,
продолжительность – не менее 24 часов...

Запрос 2: Поиск аварийных случаев

Были ли в наших отчётах случаи разгерметизации на месторождении Западное?
Найди все упоминания.

Запрос 3: Технические параметры

Какие технические характеристики насосов ЦНС-180 указаны в документации?

Запрос 4: Анализ на английском

What are the recommended intervals for pipeline inspection according to our documents?

Claude Code поддерживает запросы на русском и английском — используйте тот язык, на котором написаны ваши документы.

Запрос 5: Сравнение данных

Сравни показатели дебита скважин из отчёта за 2023 и 2024 год.
Есть ли динамика снижения?

Запрос 6: Поиск конкретного документа

Покажи, какие файлы у нас есть в базе знаний. Есть ли документы по геофизическим исследованиям скважин (ГИС)?

[>>] Что произойдёт:

Claude Code вызывает: list_documents()

Ответ:

В базе знаний найдено 47 документов. Файлы по ГИС:

- GIS_Otchet_Skvazhina_245_2024.pdf*
- Metodika_GIS_interpretaciya.docx*
- ...*

Как Claude Code решает, когда использовать RAG

Claude Code автоматически обращается к базе знаний, когда:

- Вопрос касается конкретных документов, отчётов или регламентов
- В вопросе есть слова: «в наших документах», «в регламенте», «в отчёте»
- Вопрос требует точных технических данных из вашей отрасли

Claude Code **не** обращается к RAG, когда:

- Вопрос общий («что такое давление?»)
- Вы просите написать код или текст
- Вопрос явно не требует документов

Совет: Если хотите гарантированно использовать RAG, явно попросите:

Используя нашу базу знаний, найди информацию о...

Пример полного диалога

Вы: Найди в документах требования к охране труда при работе на высоте

Claude Code: Обращаюсь к базе знаний...

[Вызов: search_knowledge_base("требования охрана труда работа на высоте")]

Найдено 3 релевантных документа:

- 1. **ОТ_Регламент_высотные_работы.pdf** (релевантность: 94%)*
 - > "При работе на высоте свыше 1.8 метра обязательно применение*
 - > страховочной привязи типа А или Б согласно ГОСТ Р 58758-2019..."*
- 2. **Инструкция_ИОТ-045-2023.docx** (релевантность: 87%)*
 - > "Перед началом высотных работ бригадир обязан провести*
 - > внеплановый инструктаж и заполнить наряд-допуск формы НД-3..."*

Итоговый ответ:

Согласно вашим корпоративным документам, основные требования следующие:

- Обязательная страховочная привязь при высоте от 1.8 м*
- Наряд-допуск формы НД-3 перед началом работ*
- Внеплановый инструктаж бригады...*

9.5 Настройка CLAUDE.md

Что такое CLAUDE.md и зачем он нужен

CLAUDE.md — это специальный файл, который Claude Code автоматически читает при запуске из определённой папки. Он задаёт **контекст и правила поведения** для AI: кто вы, чем занимаетесь, какими инструментами пользоваться по умолчанию.

Без CLAUDE.md:

- Claude не знает, что вы работаете в нефтегазовой отрасли
- Не знает, что у вас есть база знаний с документами
- Не знает предпочтительный язык ответов

С CLAUDE.md:

- Claude сразу понимает контекст проекта
- Автоматически проверяет базу знаний по профессиональным вопросам
- Отвечает на русском языке по умолчанию
- Использует правильную терминологию

Шаг 9.5.1: Создать файл ~/KMS/CLAUDE.md

```
nano ~/KMS/CLAUDE.md
```

Вставьте следующее содержимое:

```
# Контекст проекта: KMS – База знаний нефтегазового инженера

## 0 проекте
Это рабочая среда инженера-нефтяника АО «НИИ НПО «ЛУЧ» - ПФ.
Основная задача – поиск и анализ технической документации по нефтегазовой отрасли.

## Структура проекта
- `~/KMS/archive/` – архив документов (PDF, DOCX, XLSX, PPTX, TXT, CSV, ODT)
- `~/KMS/notes/` – заметки Obsidian в формате Markdown
- `~/KMS/rag_system/` – RAG-система v5.0 на базе ChromaDB

## База знаний (RAG-система)
Ты имеешь доступ к инструментам через MCP-сервер `rag_kms`.

**ВАЖНО:** При любом вопросе о нефтегазовой тематике, технических регламентах, отчётах или документации – ВСЕГДА сначала обращайся к базе знаний через `search_knowledge_base`.

Доступные инструменты:
- `rag_kms:search_knowledge_base` – основной инструмент поиска
- `rag_kms:list_documents` – список документов в базе
- `rag_kms:get_document_info` – детали конкретного файла
- `rag_kms:get_stats` – статус системы

## Тематика документов
Документы могут содержать информацию по следующим темам:
- Разработка и эксплуатация нефтяных и газовых месторождений
- Технические регламенты и инструкции по ОТ и ПБ
- Геофизические исследования скважин (ГИС)
- Гидродинамические исследования скважин (ГДИС)
- Трубопроводный транспорт нефти и газа
- Геологические отчёты и подсчёты запасов
- Буровые работы и заканчивание скважин
- Оборудование УЭЦН, ШГН, АГЗУ

## Правила работы
1. Отвечай на том языке, на котором задан вопрос (по умолчанию – русский)
2. При поиске в базе знаний используй top_k=7 для обычных вопросов, top_k=10-15 для сложных аналитических запросов
3. Всегда указывай источник (имя файла) при цитировании
4. Если информация не найдена в базе – прямо об этом сообщи, не выдумывай
5. Для проверки состояния системы используй `get_stats`

## Формат ответов
- Технические данные – со ссылкой на источник и страницу, если доступно
- Цифры и нормативы – точно, без округления
- Если найдено несколько источников – сравни их, укажи расхождения
```

Сохраните: Ctrl+O, Enter, Ctrl+X.

Как запускать Claude Code с контекстом проекта

Всегда запускайте Claude из папки ~/KMS:

```
cd ~/KMS
claude
```

При запуске вы увидите подтверждение, что CLAUDE.md загружен:

```
Loading project context from CLAUDE.md...
Context loaded: KMS – База знаний нефтегазового инженера
```

[i] Совет
Добавьте алиас в ~/.bashrc для быстрого запуска:

```
echo "alias kms='cd ~/KMS && claude'" >> ~/.bashrc
source ~/.bashrc
# Теперь можно запускать одной командой:
kms
```

9.6 Использование инструментов напрямую

Иногда удобно вызывать инструменты явно, не полагаясь на автоматическое решение Claude. Это особенно полезно для точной настройки поиска.

search_knowledge_base — основной инструмент поиска

Параметры:

Параметр	Тип	По умолчанию	Описание
query	строка	—	Поисковый запрос (обязательный)
top_k	число	7	Количество возвращаемых фрагментов
generate_answer	bool	True	Генерировать ответ через LLM
file_filter	строка	""	Фильтр по имени файла

Примеры явных вызовов:

```
Вызови search_knowledge_base с параметрами:
- query: "давление опрессовки промышленного трубопровода"
- top_k: 10
- generate_answer: true
```


Используй `search_knowledge_base` для поиска:

- `query`: "техническое обслуживание УЭЦН"
- `file_filter`: "регламент"

(искать только в файлах, в названии которых есть "регламент")

Поиск без генерации ответа (только релевантные фрагменты):

- `query`: "КВД гидродинамика"
- `top_k`: 3
- `generate_answer`: false

Когда менять `top_k`:

- `top_k=3` — быстрый поиск, достаточно для простых вопросов
- `top_k=7` — стандартный (по умолчанию)
- `top_k=10` — глубокий анализ, когда нужно охватить больше документов
- `top_k=15-20` — полный охват темы (медленнее)

[i] Совет

Кросс-языковая балансировка (RU/EN). Модель эмбедингов слегка предпочитает язык запроса, и при малом `top_k` источники на другом языке вытеснялись из выдачи (например, на русский вопрос не попадали англоязычные статьи). Поэтому поиск идёт в два этапа: достаётся широкий пул кандидатов (`RETRIEVAL_FETCH_K` = 40), затем в финальную выдачу гарантированно попадает минимум `MIN_CHUNKS_PER_LANGUAGE` = 2 фрагмента каждого из языков `BALANCED_LANGUAGES` = ("ru", "en"), а остальные места заполняются по лучшему совпадению. Балансировка «мягкая»: если по теме документы есть только на одном языке, чужой язык не навязывается. Отключается флагом `CROSS_LANGUAGE_BALANCE` = False в `config.py`. Проверить распределение языков в выдаче: `python check_cross_language.py "ваш вопрос"` (а `--stats` покажет языки в индексе).

list_documents — список всех документов

Покажи мне список всех документов в базе знаний

или явно:

Вызови `list_documents` с `limit=100`

Параметры:

Параметр	По умолчанию	Описание
limit	50	Максимальное количество файлов в выводе

[>>] Пример вывода:

Документы в базе знаний (47 файлов):

1. Регламент_T0_нефтепровод_2024.pdf (загружен: 2024-11-15)
2. GIS_Otchet_Skvazhina_245.pdf (загружен: 2024-10-03)
3. Инструкция_по_ОТ_высотные_работы.docx (загружен: 2024-09-20)
- ...

get_document_info — детали конкретного документа

Получи информацию о документе "Регламент_T0_нефтепровод_2024.pdf"

[>>] Пример вывода:

Файл: GIS_Otchet_Skvazhina_245.pdf
Размер: 4.2 МБ
Дата индексации: 2024-10-03 14:22
Количество чанков: 87
Язык: русский
Краткое содержание: Отчёт по ГИС скважины №245 Западного месторождения...

Когда использовать:

- Нужно убедиться, что конкретный файл проиндексирован
- Хотите узнать дату последнего обновления документа
- Нужно понять, насколько подробно документ разбит на фрагменты

get_stats — статистика и диагностика

Проверь состояние базы знаний

[>>] Пример вывода:

```
=== Статистика RAG-системы ===
Документов в индексе: 47
Всего фрагментов (чанков): 2847
Размер базы ChromaDB: 156 МБ
Модель эмбедингов: nomic-embed-text

=== Статус LLM-провайдера ===
Провайдер: ollama
Статус: ONLINE
Модель: qwen3.5:latest
Генерация ответов: ДОСТУПНА

Последнее обновление индекса: 2026-01-15 09:30
```

Когда использовать:

- Перед важной сессией работы — убедиться, что всё работает
- Если ответы кажутся неполными — проверить количество документов

- При подозрении, что LLM-провайдер не работает

9.7 Устранение проблем

9.7.1: ModuleNotFoundError — неверный путь к Python

Симптом:

```
Error: ModuleNotFoundError: No module named 'chromadb'
```

Причина: В `mcp_servers.json` указан неверный путь к Python — используется системный Python вместо Python из виртуального окружения.

Решение:

```
# 1. Активируйте окружение:
source ~/KMS/rag_system/.venv/bin/activate

# 2. Получите ТОЧНЫЙ путь к Python:
which python
# Вывод: /home/ivanov_ap/KMS/rag_system/.venv/bin/python

# 3. Обновите mcp_servers.json:
nano ~/.claude/mcp_servers.json
# Замените значение "command" на полученный путь

# 4. Проверьте путь вручную:
/home/ivanov_ap/KMS/rag_system/.venv/bin/python -c "import chromadb; print('OK')"
```

[>>] Ожидаемый вывод последней команды:

```
OK
```

9.7.2: Инструменты не появляются в /tools

Симптом: Команда `/tools` в Claude Code не показывает `rag_kms:*` инструменты.

Пошаговая диагностика:

```
# Шаг 1: Проверить синтаксис JSON:
cat ~/.claude/mcp_servers.json | python3 -m json.tool

# Шаг 2: Проверить, что файл находится в правильном месте:
ls -la ~/.claude/mcp_servers.json

# Шаг 3: Проверить права на чтение файла:
chmod 644 ~/.claude/mcp_servers.json

# Шаг 4: Тест запуска сервера вручную:
source ~/KMS/rag_system/.venv/bin/activate
python ~/KMS/rag_system/mcp_rag_server.py
# Если появляются ошибки – исправьте их (см. п. 9.7.1)
```

После исправлений **полностью перезапустите Claude Code** (выйдите через /exit и запустите снова).

9.7.3: База знаний пуста

Симптом:

```
get_stats вернул: Документов в индексе: 0
```

Причина: Документы ещё не были проиндексированы.

Решение:

```
source ~/KMS/rag_system/.venv/bin/activate
cd ~/KMS/rag_system/

# Запустить индексацию документов:
python ingest.py
```

После индексации проверьте через `get_stats` — количество документов должно увеличиться.

9.7.4: LLM-провайдер недоступна

Симптом:

```
get_stats вернул: Статус: OFFLINE
Генерация ответов: НЕДОСТУПНА
```

Это не критично: система продолжит работать, поиск будет возвращать релевантные фрагменты без итогового LLM-ответа.

Решение для Ollama:

```
# Проверить статус Ollama:
ollama list

# Если Ollama не запущена — запустить:
ollama serve &

# Проверить, что нужные модели загружены:
ollama list
```

Решение для облачного провайдера (Groq/DeepSeek/OpenRouter):

1. Проверьте интернет-соединение: `ping api.groq.com`
2. Проверьте правильность API-ключа в `config.py` или переменных окружения
3. Временно переключитесь на ollama в `config.py`

9.7.5: Claude Code не видит `mcp_servers.json`

Симптом: Файл создан, но Claude Code его игнорирует.

Диагностика:

```
# Проверить точное расположение:
ls -la ~/.claude/
# Файл должен называться именно mcp_servers.json

# Проверить содержимое:
cat ~/.claude/mcp_servers.json

# Убедиться, что ~/.claude — это папка, а не файл:
file ~/.claude
# Должно быть: /home/USERNAME/.claude: directory
```

Частая ошибка: Файл создан как `~/.claude` (без слеша) вместо `~/.claude/mcp_servers.json`. Проверьте, что `~/.claude` — это именно **папка**.

9.7.6: Медленный первый запуск

Симптом: Первый запрос после старта Claude Code занимает 30-60 секунд.

Это нормально. При первом обращении к базе знаний происходит:

1. Загрузка модели эмбедингов в оперативную память (~500 МБ)
2. Подключение к ChromaDB
3. Инициализация соединения с LLM-провайдером

Все последующие запросы в рамках одной сессии будут значительно быстрее (2-5 секунд).

Совет: Не нажимайте Ctrl+C во время ожидания — дайте системе время на инициализацию.

9.8 Шпаргалка команд Claude Code

Действие	Команда
Установка	
Установить Claude Code	<code>npm install -g @anthropic-ai/claude-code</code>
Обновить Claude Code	<code>npm update -g @anthropic-ai/claude-code</code>
Авторизация	<code>claude login</code>
Проверить версию	<code>claude --version</code>
Запуск	
Запустить Claude Code (обычно)	<code>claude</code>
Запустить из папки KMS (рекомендуется)	<code>cd ~/KMS && claude</code>
Выйти из Claude Code	<code>/exit</code>
Диагностика в Claude Code	
Проверить доступные инструменты	<code>/tools</code> (внутри Claude Code)
Проверить статус системы	Спросить: <code>Вызови get_stats</code>
Конфигурация	
Открыть конфиг MCP	<code>nano ~/.claude/mcp_servers.json</code>
Проверить синтаксис JSON	<code>cat ~/.claude/mcp_servers.json \ python3 -m json.tool</code>
Показать USERNAME	<code>echo \$USER</code>
Узнать путь к Python	<code>source ~/KMS/rag_system/.venv/bin/activate && which python</code>
Проверить импорт зависимостей	<code>python -c "import chromadb, ollama; print('OK')"</code>
MCP-сервер	
Тестовый запуск сервера	<code>source ~/KMS/rag_system/.venv/bin/activate && python ~/KMS/rag_system/mcp_rag_server.py</code>
Логи и отладка	
Запуск Claude с отладкой	<code>claude --debug</code>

Приложение А. Структура файлов системы

Для тех, кто хочет понять, что за что отвечает:

~/KMS/	
├─ CLAUDE.md	← Контекст проекта для Claude Code (Раздел 9)
├─ archive/	→ Ваши документы (или символическая ссылка на USB)
│ ├─ article_1.pdf	→ PDF-документ
│ ├─ report.docx	→ Word-документ
│ ├─ data.xlsx	→ Таблица Excel
│ ├─ presentation.pptx	→ Презентация
│ └─ CO2_storage/	→ Можно организовывать по подпапкам
│ └─ co2_paper.pdf	
├─ notes/	→ Хранилище Obsidian
│ ├─ <заголовок-документа>.md	→ Заметки создаются прямо в notes/ (имя – слог заголовка)
│ ├─ ...	→ по одной заметке на документ
│ └─ Templates/	→ Шаблоны для заметок
│ ├─ Literature Note.md	
│ ├─ RAG Query.md	
│ ├─ Research Session.md	
│ └─ Shell Commands.md	
└─ rag_system/	→ Сама система
│ ├─ config.py	→ Настройки (пути, провайдер, модель, форматы)
│ ├─ llm_provider.py	→ Модуль мульти-провайдерного LLM (v5.0, НОВЫЙ)
│ ├─ ingest.py	→ Индексация документов всех форматов в ChromaDB
│ ├─ doc_to_obsidian.py	→ Создание заметок Obsidian (все форматы)
│ ├─ pdf_to_obsidian.py	→ Старый скрипт (только PDF, для совместимости)
│ ├─ watcher.py	→ Наблюдатель за папкой (все форматы)
│ ├─ rag_engine.py	→ Движок поиска и ответов
│ ├─ chat_ui.py	→ Веб-интерфейс чата
│ ├─ mcp_rag_server.py	→ MCP-сервер для Claude Code (Раздел 9)
│ ├─ requirements.txt	→ Список Python-пакетов
│ ├─ install_service.sh	→ Скрипт установки systemd-сервиса
│ ├─ chroma_db/	→ База данных векторных представлений
│ ├─ logs/	→ Журналы работы системы
│ │ └─ rag_system.log	
│ └─ .venv/	→ Виртуальное окружение Python
│ └─ bin/	
│ └─ activate	→ Скрипт активации окружения
~/./claude/	→ Конфигурация Claude Code (Раздел 9)
└─ mcp_servers.json	→ Конфигурация MCP

Приложение Б. Глоссарий

API-ключ — уникальный секретный код для аутентификации при обращении к облачному сервису (Groq, DeepSeek, OpenRouter и т.д.). Выдаётся в личном кабинете сервиса. Никому не передавайте.

ChromaDB — база данных для хранения и поиска векторных представлений (эмбеддингов). В отличие от обычных баз данных, умеет находить «похожие по смыслу» записи, а не только точные совпадения.

Claude Code — AI-ассистент компании Anthropic, работающий в командной строке. Умеет вызывать внешние инструменты через протокол MCP, в том числе вашу RAG-систему.

Embedding (эмбеддинг, векторное представление) — числовой вектор (набор чисел), кодирующий смысл текстового фрагмента. Тексты с похожим смыслом имеют близкие векторы. Именно это позволяет системе находить документы о «поглощении углекислого газа», если вы спрашиваете про «захоронение CO₂».

LLM (Large Language Model, большая языковая модель) — программа, обученная на огромных объёмах текста, способная понимать и генерировать человеческий язык. Примеры: GPT-4, Claude, Mistral, Qwen, LLaMA.

LLM-провайдер — сервис или программа, предоставляющие доступ к языковой модели. В RAG KMS v5.0 поддерживаются: ollama (локально), groq, deepseek, openrouter, lmstudio.

Markdown — простой язык разметки текста с помощью специальных символов. Например, ****жирный** жирный**, **# Заголовок большой заголовок**. Obsidian использует Markdown для всех заметок.

MCP (Model Context Protocol) — открытый протокол от Anthropic, позволяющий AI-ассистентам (в частности, Claude Code) подключаться к внешним инструментам и источникам данных. Работает через JSON-RPC поверх stdio.

Ollama — программа для локального запуска языковых моделей без интернета.

OCR (Optical Character Recognition) — технология распознавания текста на изображениях. Нужна для работы со сканированными PDF.

pip — менеджер пакетов Python. Позволяет устанавливать библиотеки командой `pip install`.

RAG (Retrieval-Augmented Generation) — технология, при которой языковая модель сначала ищет релевантные фрагменты в базе знаний, а затем использует их для генерации ответа. Без RAG модель отвечает только на основе своих «знаний» из обучения; с RAG — опирается на ваши конкретные документы.

Systemd / systemctl — система управления сервисами в Linux. `systemctl` — утилита для управления сервисами.

Symlink (символическая ссылка) — файл, который является «указателем» на другой файл или папку в файловой системе. Как ярлык в Windows, но более «настоящий» — программы работают с ним как с реальным файлом.

Vault (хранилище Obsidian) — папка на диске, которую Obsidian использует для хранения заметок и отслеживания связей между ними.

Virtual environment (виртуальное окружение Python) — изолированная среда Python со своим набором установленных пакетов, независимая от системного Python.

Приложение В. Проверочный список установки

Используйте этот список, чтобы убедиться, что все шаги выполнены правильно:

Базовая система (обязательно):

- [] Ubuntu 20.04/22.04/24.04 установлена и обновлена
- [] Python 3.10–3.12 доступен (`python3 --version`; на 3.13+ ChromaDB не соберётся — нужен 3.12)
- [] Ollama установлена (`ollama list` не выдаёт ошибку)
- [] Языковая модель скачана (видна в `ollama list`)
- [] Папка `~/KMS/archive` создана (или символическая ссылка на USB)
- [] Архив системы распакован в `~/KMS/rag_system/`
- [] Файл `llm_provider.py` присутствует в `~/KMS/rag_system/`
- [] Виртуальное окружение создано (`~/KMS/rag_system/.venv/`)
- [] Зависимости установлены (`pip install -r requirements.txt`)
- [] Пути в `config.py` настроены
- [] LLM-провайдер и модель в `config.py` выбраны
- [] Obsidian установлен
- [] Хранилище Obsidian открыто (`~/KMS/notes/`)
- [] Шаблоны скопированы в `~/KMS/notes/Templates/`
- [] Первые документы добавлены в архив
- [] Индексация прошла успешно (`python ingest.py`)
- [] Заметки Obsidian созданы (`python doc_to_obsidian.py`)
- [] Чат-интерфейс открывается в браузере (`http://127.0.0.1:7860`)
- [] Сервис наблюдателя установлен и запущен (`systemctl --user status rag-watcher`)

Мульти-провайдер (если используете облачные LLM):

- [] API-ключ получен (Groq / DeepSeek / OpenRouter)
- [] Ключ добавлен в `~/.bashrc` или `config.py`
- [] Провайдер указан в `config.py` (поле `LLM_PROVIDER`)
- [] Модель для провайдера указана (поле `LLM_MODEL`)
- [] Соединение с провайдером протестировано

Claude Code + MCP (опционально, Раздел 9):

- [] Node.js 18+ установлен (`node --version`)
- [] Claude Code установлен (`claude --version`)
- [] Авторизация выполнена (`claude login`)
- [] Папка `~/.claude/` создана
- [] Файл `~/.claude/mcp_servers.json` создан с правильными путями
- [] Синтаксис JSON проверен (`cat ~/.claude/mcp_servers.json | python3 -m json.tool`)
- [] Инструменты RAG видны в `/tools` внутри Claude Code
- [] Файл `~/KMS/CLAUDE.md` создан с контекстом проекта

Приложение Г. Часто задаваемые вопросы (FAQ)

Q: Нужен ли постоянный интернет для работы системы?

A: Нет. После установки всех компонентов и скачивания моделей система работает полностью офлайн (если выбран провайдер ollama или lmstudio). При использовании облачных провайдеров (groq, deepseek, openrouter) интернет необходим для каждого запроса.

Q: Можно ли использовать другие языковые модели, не только Mistral?

A: Да! В версии 5.0 доступны 5 провайдеров. Для Ollama подойдёт любая модель из каталога ollama.com/library. Рекомендуется: qwen2.5:7b (локально, лучший для данного проекта), llama-3.3-70b-versatile (Groq, бесплатно, мощнее). Скачайте модель через `ollama pull ИМЯ_МОДЕЛИ` и укажите её в `config.py`. Подробнее — в Разделе 7.

Q: Что если PDF на нескольких языках (например, русский текст + английские таблицы)?

A: Система корректно обработает смешанный текст. Языковая модель также поймёт смешанный контент. Теги будут создаваться на основе содержания.

Q: Сколько максимум документов может хранить система?

A: Практического ограничения нет. ChromaDB хорошо масштабируется. Реальные ограничения — место на диске (база занимает примерно 50% от размера документов) и оперативная память при поиске. При 10 000+ документах поиск может занимать 3-5 секунд вместо обычных 0.5-1 секунды.

Q: Система потеряла индекс. Нужно ли снова обрабатывать все документы?

A: Да, если папка `chroma_db/` была удалена или повреждена. Просто запустите `python ingest.py` — система обработает все документы, которые сейчас есть в `~/KMS/archive/`. Это займёт время, но всё восстановится.

Q: Можно ли запускать чат из нескольких компьютеров в сети?

A: Да. Измените в `chat_ui.py` строку `launch()` на `launch(server_name="0.0.0.0")` — интерфейс станет доступен по IP-адресу вашего компьютера в локальной сети: `http://192.168.x.x:7860`.

Q: Как сделать резервную копию базы знаний?

A: Скопируйте папку `~/KMS/rag_system/chroma_db/` на резервный диск. Также не забудьте скопировать `~/KMS/notes/` (заметки Obsidian) и `~/KMS/rag_system/config.py` (настройки).

Q: Как переключиться с Ollama на Groq и обратно?

A: Откройте `config.py` и измените три строки: `LLM_PROVIDER`, `LLM_MODEL`, `LLM_API_KEY`. Перезапустите чат или скрипт. Подробные примеры — в Разделе 7.1.

Q: Мой API-ключ Groq закончился на сегодня. Что делать?

A: Groq бесплатный тариф — 14 400 запросов в день. Дождитесь сброса (в полночь UTC) или временно переключитесь на ollama: в `config.py` измените `LLM_PROVIDER: str = "ollama"`.

После сброса лимита верните обратно.

Q: Зачем нужен Claude Code, если уже есть chat_ui.py?

A: chat_ui.py — это простой чат-интерфейс: вы задаёте вопрос, получаете ответ. Claude Code — более мощный инструмент: он умеет работать с несколькими инструментами одновременно, читать файлы, выполнять команды, строить многоэтапные рассуждения. Например, Claude Code может найти информацию в базе знаний, потом открыть конкретный файл, потом написать резюме — всё в одном разговоре. Подробнее — Раздел 9.

Q: Как безопасно хранить API-ключи, не вставляя их в config.py?

A: Используйте переменные окружения в ~/.bashrc (подробности — Раздел 7.3). Поле LLM_API_KEY в config.py можно оставить пустым — llm_provider.py автоматически прочитает переменную окружения RAG_GROQ_API_KEY, RAG_DEEPSEEK_API_KEY или RAG_OPENROUTER_API_KEY.

Шпаргалка. Все ключевые команды

Эта таблица содержит все основные команды в одном месте. Рекомендуем распечатать её или сохранить в отдельном файле.

Основные операции RAG-системы

Действие	Команда
Запустить чат	<code>cd ~/KMS/rag_system && source .venv/bin/activate && python chat_ui.py</code>
Открыть чат в браузере	<code>http://127.0.0.1:7860</code>
Статус наблюдателя	<code>systemctl --user status rag-watcher</code>
Перезапустить наблюдатель	<code>systemctl --user restart rag-watcher</code>
Обновить индекс вручную	<code>cd ~/KMS/rag_system && source .venv/bin/activate && python ingest.py</code>
Обновить заметки Obsidian	<code>cd ~/KMS/rag_system && source .venv/bin/activate && python doc_to_obsidian.py</code>
Вопрос без браузера	<code>cd ~/KMS/rag_system && source .venv/bin/activate && python rag_engine.py "вопрос"</code>
Статистика базы	<code>cd ~/KMS/rag_system && source .venv/bin/activate && python rag_engine.py --stats</code>
Активировать окружение	<code>source ~/KMS/rag_system/.venv/bin/activate</code>

Управление сервисом наблюдателя

Действие	Команда
Статус	<code>systemctl --user status rag-watcher</code>
Запустить	<code>systemctl --user start rag-watcher</code>
Остановить	<code>systemctl --user stop rag-watcher</code>
Перезапустить	<code>systemctl --user restart rag-watcher</code>
Включить автозапуск	<code>systemctl --user enable rag-watcher</code>
Отключить автозапуск	<code>systemctl --user disable rag-watcher</code>
Последние 50 строк лога	<code>journalctl --user -u rag-watcher -n 50</code>
Лог в реальном времени	<code>journalctl --user -u rag-watcher -f</code>

Управление Ollama и моделями

Действие	Команда
Список установленных моделей	<code>ollama list</code>
Скачать Mistral	<code>ollama pull mistral</code>
Скачать Qwen2.5 7B (рекомендуется)	<code>ollama pull qwen2.5:7b</code>
Скачать Qwen2.5 14B (мощнее)	<code>ollama pull qwen2.5:14b</code>
Скачать LLaMA 3.1 8B	<code>ollama pull llama3.1:8b</code>
Скачать DeepSeek R1 7B	<code>ollama pull deepseek-r1:7b</code>
Скачать компактную модель	<code>ollama pull mistral:7b-instruct-q4_0</code>
Обновить Ollama	<code>curl -fsSL https://ollama.com/install.sh sh</code>
Проверить версию Ollama	<code>ollama --version</code>
Удалить модель	<code>ollama rm mistral</code>
Статус запущенных моделей	<code>ollama ps</code>
Запустить сервер Ollama	<code>ollama serve</code>
Проверить API Ollama	<code>curl http://localhost:11434/api/tags</code>

Управление LLM-провайдерами

Действие	Описание
Переключить на Ollama	B config.py: LLM_PROVIDER="ollama", LLM_MODEL="qwen2.5:7b"
Переключить на Groq	B config.py: LLM_PROVIDER="groq", LLM_MODEL="llama-3.3-70b-versatile", LLM_API_KEY="gsk_..."
Переключить на DeepSeek	B config.py: LLM_PROVIDER="deepseek", LLM_MODEL="deepseek-chat", LLM_API_KEY="sk-..."
Переключить на OpenRouter	B config.py: LLM_PROVIDER="openrouter", LLM_MODEL="qwen/qwen3-14b:free", LLM_API_KEY="sk-or-..."
Переключить на LM Studio	B config.py: LLM_PROVIDER="lmstudio", LLM_MODEL="qwen2.5-7b-instruct"
Задать ключ Groq через env	export RAG_GROQ_API_KEY="gsk_..." (в ~/.bashrc)
Задать ключ DeepSeek через env	export RAG_DEEPSEEK_API_KEY="sk-..." (в ~/.bashrc)
Задать ключ OpenRouter через env	export RAG_OPENROUTER_API_KEY="sk-or-..." (в ~/.bashrc)

Диагностика и обслуживание

Действие	Команда
Место на дисках	df -h
Размер базы ChromaDB	du -sh ~/KMS/rag_system/chroma_db/
Размер архива документов	du -sh ~/KMS/archive/
Количество PDF в архиве	find ~/KMS/archive -name "*.pdf" \ wc -l
Все документы по типам	find ~/KMS/archive -type f \ sed 's/.*\\.//' \ sort \ uniq -c
Версия Python	python3 --version
Логи системы (файл)	tail -f ~/KMS/rag_system/logs/rag_system.log
Статус USB-ссылки	ls -la ~/KMS/archive
Топ-20 частых тегов	grep -r "auto/" ~/KMS/notes/ \ grep "tags:" \ sort \ uniq -c \ sort -rn \ head -20
Очистить кеш pip	pip cache purge
Очистить старые журналы	journalctl --user --vacuum-size=100M

Claude Code и MCP (Раздел 9)

Действие	Команда
Установить Claude Code	<code>npm install -g @anthropic-ai/claude-code</code>
Обновить Claude Code	<code>npm update -g @anthropic-ai/claude-code</code>
Авторизация	<code>claude login</code>
Проверить версию	<code>claude --version</code>
Запустить с контекстом проекта	<code>cd ~/KMS && claude</code>
Выйти из Claude Code	<code>/exit</code>
Проверить инструменты RAG	<code>/tools</code> (внутри Claude Code)
Открыть конфиг MCP	<code>nano ~/.claude/mcp_servers.json</code>
Проверить синтаксис MCP-конфига	<code>cat ~/.claude/mcp_servers.json \ python3 -m json.tool</code>
Тест MCP-сервера вручную	<code>source ~/KMS/rag_system/.venv/bin/activate && python ~/KMS/rag_system/mcp_rag_server.py</code>
Запуск с отладкой	<code>claude --debug</code>
Быстрый запуск (псевдоним)	<code>kms</code> (после <code>echo "alias kms='cd ~/KMS && claude'" >> ~/.bashrc</code>)

Система работает полностью локально при использовании провайдеров ollama и lmstudio. Данные не покидают ваш компьютер. При использовании облачных провайдеров (groq, deepseek, openrouter) поисковые запросы и контекст документов передаются на серверы провайдера — используйте с учётом политики конфиденциальности вашей организации.

Версия документа: 5.1 (уточнено по опыту эксплуатации) | **Год:** 2026
Автор: Александр Князев, начальник отдела технологий декарбонизации АО «НИИ НПО «ЛУЧ» — ПФ
Поддерживаемые форматы документов: PDF, DOCX, DOC, TXT, MD, XLSX, CSV, PPTX, ODT

Успехов в работе с системой управления знаниями!

Александр Князев, начальник отдела технологий декарбонизации АО «НИИ НПО «ЛУЧ» — ПФ